Batch-3

AI MINI CAPSTONE PROJECT REPORT

on

"Retrieval-Augmented Generation (RAG) Chatbot for College Regulations and Syllabus Queries"

Submitted

by

251FA04D58 *(Team Lead)*
Vura Nagamani Susmitha

251FA04D60
Vuyyuru Hasini

251FA04D62
Vuyyuru Madhuri

251FA04E70
Chatla Barghava Reddy

Under the guidance of
Mrs.SD.Shareefunnisa
Assistant Professor, CSE



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
VIGNAN'S FOUNDATION FOR SCIENCE, TECHNOLOGY AND RESEARCH
Deemed to be UNIVERSITY
Vadlamudi, Guntur.
ANDHRA PRADESH, INDIA, PIN-522213.

Scanned with OKEN Scanner

CERTIFICATE

This is to certify that the AI Mini Capstone Project entitled "Retrieval-Augmented Generation (RAG) Chatbot for College Regulations and Syllabus Queries" that is being submitted by 251FA04D58 (Vura Nagamani Susmitha), 251FA04D60 (Vuyyuru Hasini), 251FA04D62 (Vuyyuru Madhuri), 251FA04E70 (Chatla Barghava Reddy) for partial fulfilment of the AI Mini Capstone Project is a bonafide work carried out under the supervision of Mrs.SD. Shareefunnisa, Assistant Professor, Department of CSE.

HOD, CSE

Mrs.SD.Shareefunnisa
Assistant Professor

Scanned with OKEN Scanner

DECLARATION

We hereby declare that the AI Mini Capstone Project entitled "Retrieval-Augmented Generation (RAG) Chatbot for College Regulations and Syllabus Queries" is being submitted by 251FA04D58 (Vura Nagamani Susmitha), 251FA04D60 (Vuyyuru Hasini), 251FA04D62 (Vuyyuru Madhuri), 251FA04E70 (Chatla Barghava Reddy) for partial fulfilment of AI Mini Capstone Project course work. This is our original work, and this project has not formed the basis for the award of any degree. We have worked under the supervision of Mrs.SD.Shareefunnisa, Assistant Professor, Department of CSE, VFSTR.

By

251FA04D58 (Vura Nagamani Susmitha) V. Susmitha

251FA04D60 (Vuyyuru Hasini) V. Hasini

251FA04D62 (Vuyyuru Madhuri) V. Madhuri

251FA04E70 (Chatla Barghava Reddy) Ch. Bhargav

Date:

Scanned with OKEN Scanner

Contents

Abstract 3
1. Introduction 3
2. Problem Statement 4
3. Motivation 5
4. Objectives 5
5. Existing Approach 5
6. Proposed Methodology 6
7. Core Components of the Hybrid Retrieval Algorithm 8
8. PROPOSED HYBRID RETRIEVAL ALGORITHM 12
A. Algorithm overview and B. Objective 12
C. Input and output; D. Technologies and techniques 12
E. Detailed working procedure 13
F. Mathematical explanation 14
G. Pseudocode 16
H. Step-by-step execution and flowchart 17
I. Worked example 18
J. Advantages; K. Why this algorithm was selected 19
9. System Architecture 20
10. Implementation 21
11. Results and Discussion 22
12. Real-World Applications 27
13. Limitations 27
14. Future Scope 27
15. Conclusion 27
16. References 28
17. Appendix A: Source Code 30

Scanned with OKEN Scanner

Abstract

Students in a college often have doubts about attendance rules, grading, CGPA, backlogs, exams, fees and the syllabus of each subject. This information is spread over long PDF files and notice boards, so it takes time to find, and the same questions are asked again and again to teachers and the college office.

This project builds a chatbot that answers such questions automatically using Retrieval-Augmented Generation (RAG). The chatbot first searches a knowledge base for the stored question that is closest to the student's question and then returns the answer stored for it. Because the answer comes from the knowledge base and is not guessed, the chatbot does not make up wrong information.

The central technical contribution of the project is its retrieval algorithm: the Hybrid Retrieval-Based RAG Algorithm using TF-IDF, Character N-Grams and MiniLM Sentence Embeddings . The knowledge base is a CSV file of 5,000 questions and answers. For every new question the algorithm scores all stored questions in three complementary ways: (1) word matching with TF-IDF, (2) letter-group matching with character N-Grams, which tolerates spelling mistakes, and (3) meaning matching with MiniLM sentence embeddings searched through FAISS. The three scores are combined, the stored question with the highest combined score is selected, and its answer is shown only if the score reaches the confidence threshold of 0.35. Otherwise the chatbot says that it does not know.

As reported by the team from its Google Colab experiments, the chatbot was tested with 800 questions containing spelling mistakes and different wording. Word matching alone gave 76.9% Top-1 accuracy and letter matching alone gave 97.2%, while the combined method gave 97.6% Top-1 and 98.4% Top-3 accuracy at about 3 milliseconds per question. With the threshold of 0.35, all 60 unrelated questions were refused and all 300 college questions were answered.

Keywords: Artificial Intelligence, RAG, Hybrid Retrieval, TF-IDF, Character N-Grams, Sentence Embeddings, MiniLM, FAISS, Cosine Similarity, College Regulations.

1. Introduction

A chatbot is a program that talks with a user and answers questions. Today, many chatbots are built on Large Language Models (LLMs) such as ChatGPT. These models are based on the Transformer architecture, which uses a mechanism called attention to understand how the words of a sentence are related to each other.

LLMs have one problem. They know only the data they were trained on. They do not know the rules of our college, and if we ask them, they may give a confident but wrong answer. This is called hallucination . For questions such as "Can I write the exam with 72% attendance?", a wrong answer can mislead a student, so the answer must come from the official rules and not from guessing.

1.1 What is RAG?

RAG stands for Retrieval-Augmented Generation. It has two parts:

- Retrieval: the system first searches a knowledge base and finds the information related to the question.
- Generation: the answer is then prepared from that information. It can be shown directly or rewritten by an LLM in a friendly way.

In simple words, RAG is like an open-book exam. Instead of answering from memory, the chatbot opens the book (our CSV file), finds the correct page and then answers. In this project the answer is shown exactly as stored in the CSV, so the quality of the chatbot depends mainly on how well the retrieval step finds the correct stored question. This is why the retrieval algorithm is the heart of this report.

Scanned with OKEN Scanner

1.2 Why college regulations and syllabus?

Every semester, new students ask the same questions: What is the minimum attendance? How is CGPA calculated? When are supplementary exams? Which topics come in Unit 3 of Operating Systems? A chatbot can answer these at any time of the day, and gives the same answer to every student.

1.3 How does a Transformer understand a sentence?

Take the sentence: "The student paid the fee because it was due." To understand what "it" means, a model must look at the other words. A Transformer does this using attention : for every word, it checks how much every other word matters and gives each a weight. Here, "it" pays most attention to "fee". Because the Transformer looks at the whole sentence at the same time, it understands the meaning better than older models that read one word at a time.

In this project we do not train a Transformer. We use a ready-made small Transformer (MiniLM) that has already learned the meaning of sentences, and we use it as one of the three parts of the retrieval algorithm to compare the student's question with the stored questions.

1.4 Scope of this report

This report describes the complete college RAG chatbot, but it concentrates on the retrieval algorithm. Chapter 7 explains each technique used in the algorithm, Chapter 8 presents the complete step-by-step algorithm with mathematics, pseudocode, a flowchart and a worked example, and Chapter 11 presents the reported results.

2. Problem Statement

Information about college rules and syllabus is spread across many documents. Students cannot find answers quickly, and staff spend time answering the same questions again and again. General chatbots cannot be trusted for college-specific rules because they may guess.

> **Problem definition**
> To design and develop an AI chatbot that takes a question from a student about college regulations or syllabus, finds the correct answer from a knowledge base using RAG, and says "not found" when the question is outside the knowledge base.

2.1 Input and output

Table 2: Input, process and output

Part	Details
Input	A question typed by the student, for example: "What is the minimum attendance for exams?" (spelling mistakes are allowed).
Process	Clean the question, search the 5,000 stored questions, pick the best match, check the match score.
Output	The answer, the matching question and a confidence score. If there is no good match: a "not found" message.

2.2 Main problems solved

- Students cannot find rules quickly in long documents.
- The same questions are asked again and again.
- Students type with spelling mistakes or use different words.

Scanned with OKEN Scanner

- A chatbot must not give wrong answers about attendance or exams.
- Questions that are not about college should not get an answer.

2.3 Assumptions and limitations

- The knowledge base is correct and up to date.
- Questions are in English.
- The CSV used in this project is a sample dataset. For real use, the college's own regulations must be added.

3. Motivation

The motivation for this project comes from four everyday observations in our college.

- Scattered information. Attendance, grading, CGPA, backlog, exam, fee, hostel and library rules, and the syllabus of 16 subjects, are present in different documents, so a student has to search several places for one answer.
- Repeated questions. Teachers and the college office answer the same questions every semester. A chatbot available at any time reduces this repeated work and gives the same answer to every student.
- Real typing behaviour. Students do not copy the exact wording of the rules. They make spelling mistakes ("attendence"), use short forms and ask the same thing in different words. A useful chatbot must handle this.
- Trust. A wrong answer about attendance or exams can have a real effect on a student. This motivated us to return only stored answers and to refuse questions that the chatbot cannot match with enough confidence.

These needs cannot be met by one simple search method. Exact word search breaks on spelling mistakes, and an LLM alone may guess. This motivated the design of ahybrid retrieval algorithm that combines word, letter and meaning matching, and which is the main technical contribution of the project. Academically, the project also gave us hands-on experience with LLMs, Transformers and RAG.

4. Objectives

The main objectives of this project are:

1. To understand how LLMs, Transformers and RAG work.
2. To create a CSV knowledge base of 5,000 questions and answers about college regulations and syllabus.
3. To design a hybrid retrieval algorithm that combines TF-IDF word matching, character N-Gram matching and MiniLM sentence embeddings with FAISS, and to build the chatbot in Python (Google Colab) on top of it.
4. To make the chatbot work even when the question has spelling mistakes or different wording.
5. To make the chatbot say "not found" for questions it does not know, using a confidence threshold.
6. To test the chatbot and measure its accuracy and speed.
7. To compare our combined method with simple word matching.
8. To prepare the report, PPT and A5 poster.

Scanned with OKEN Scanner

5. Existing Approach

Before selecting our method, we studied three common ways of building a question-answering chatbot for college information.

Normal keyword search. The question is compared with stored text using exact words. It is simple and fast, but it fails when words are spelt wrongly or changed. In our test it reached only 76.9% Top-1 accuracy.

Only an LLM (like ChatGPT)The model answers from what it learned during training. It does not know our college rules and may give confident but wrong answers.

Training our own LLM.A model trained on our rules would know them, but it needs very costly computers and has to be trained again whenever the rules change.

Table 3: Comparison of approaches

Method	Problem
Normal keyword search	Fails when words are spelt wrongly or changed. Accuracy was only 76.9%.
Only an LLM (like ChatGPT)	Does not know our college rules, may give wrong answers.
Training our own LLM	Needs very costly computers and has to be repeated when rules change.
RAG with hybrid search (our method)	Accurate, fast, gives answers only from the CSV, and easy to update by editing the CSV.

The gap in the existing approaches is clear: we need a method that is robust to spelling and wording changes, uses only verified answers, is cheap to run and easy to update. The proposed hybrid retrieval algorithm in Chapter 8 is designed to fill this gap.

6. Proposed Methodology

The project was done in the following steps.

1. Problem identification: selected the college chatbot problem and got the topic approved.
2. Data preparation: created the CSV file with 5,000 questions and answers in 12 categories.
3. Study: learned how RAG, TF-IDF and sentence embeddings work and selected the hybrid retrieval method.
4. Pre-processing: cleaned all the questions.
5. Index building: converted all questions into numbers (word, letter and meaning).
6. Question answering: for each new question, find the best match and check the score.
7. Testing: tested with 800 questions having spelling mistakes and different wording.
8. Results and documentation: prepared charts, report, PPT and poster.

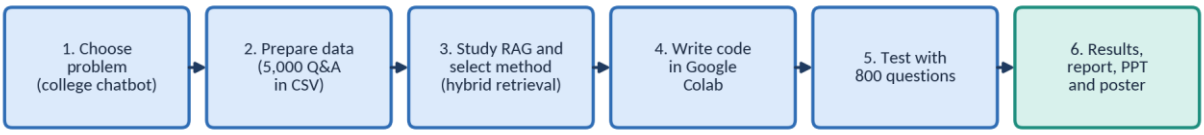


Figure 1: Project workflow

6.1 Dataset

The dataset is a CSV file named college_rag_qa_5000.csv . It has 5,000 rows and the columns id , category , fact_id , question and answer . The rows are about attendance, grading, CGPA, backlogs, exams, fees, hostel, library, placements, academics, the AI Mini-Capstone and the syllabus of 16 subjects.

Scanned with OKEN Scanner

The same fact is written in many different ways, because students ask the same thing in different words. The fact_id column is used only to check whether the chatbot found the correct fact during testing; it is not used for retrieval.

Table 4: A few rows of the CSV dataset

id	category	question	answer
2585	Syllabus	Can you tell me which subject covers exceptions?	Exceptions is covered in Unit 4 of Python Programming (25CS101).
4339	Syllabus	Quick question: unit 5 topics of DSA?	Unit 5 of Data Structures and Algorithms covers: Dynamic programming, greedy algorithms, backtracking, NP completeness.
4557	Syllabus	Could you help me understand where can I study firewalls in the syllabus?	Firewalls is covered in Unit 5 of Computer Networks (25CS304).
3439	Syllabus	In which course and unit is greedy algorithms taught?	Greedy algorithms is covered in Unit 5 of Data Structures and Algorithms (25CS201).
738	Syllabus	Kindly let me know what is taught in the 2nd unit of Artificial Intelligence?	Unit 2 of Artificial Intelligence covers: BFS, DFS, uniform cost search, iterative deepening.
2677	Syllabus	Which subject covers threats?	Threats is covered in Unit 1 of Cyber Security (25CS405).

One sample question and answer from each category is shown in Table 5 (the CGPA sample row of the original table could not be recovered, so it is left out; CGPA questions are covered in the test cases in Chapter 11), and the number of rows in each category is shown in Table 6.

Table 5: Sample from each category

Category	Sample question	Sample answer
Academics	Could you help me understand is a branch change allowed?	Branch change is allowed at the end of the first year for students with CGPA >= 8.0, ...
Attendance	Could you help me understand do I get attendance for representing the college in events?	Students officially representing the college in sports, NCC, NSS or cultural events r...
Backlogs	Quick question: is there an improvement exam option?	Students may appear for improvement exams in up to 4 theory courses in the final year...
Exams	I want to know what are the rules in the exam hall?	Mobile phones, smart watches, notes and unauthorized material are strictly prohibited...
Fees	When should I pay tuition fees?	Tuition fees must be paid before the start of every semester as per the academic cale...
Grading	Kindly let me know is the grading relative or absolute?	Grades are assigned using relative grading based on class performance, provided the m...
Hostel	Quick question: what is the hostel intime?	Hostel in-time is 8:00 PM for first years and 9:00 PM for others; late entry needs wr...
Library	What if I lose a library book?	The late fine is Rs. 2 per day per book; a lost book must be replaced or its current ...
Placements	Quick question: what is the placement policy for multiple offers?	Under the One-Student-One-Offer policy, a student may hold one offer, and a higher-ti...
Project	Quick question: when is the final review?	Batch finalization 09-10-2026, Review-1 on 12-10-2026, Review-2 on 26-10-2026, Review...
Syllabus	I want to know which subject covers routing algorithms?	Routing algorithms is covered in Unit 3 of Computer Networks (25CS304).

Scanned with OKEN Scanner

Table 6: Rows in each category

Category	Rows	Category	Rows
Syllabus	3596	Project	56
CGPA	385	Fees	56
Attendance	259	Placements	56
Grading	186	Library	42
Academics	140	Hostel	28
Exams	126	Backlogs	70
Total	5000		

7. Core Components of the Hybrid Retrieval Algorithm

The chatbot uses RAG with a hybrid retriever . "Hybrid" means that different ways of searching are combined, because each one has its own strength and its own weakness. Before any searching, every question is cleaned: it is made lower case and extra words such as "please tell me" and special symbols are removed. This is done for the stored questions and for the student's question, so that both are in the same format. For example, "I have 72% attendance, can I write the exam?" becomes "i have 72% attendance can i write the exam".

Table 7 gives an overview of the nine components of the algorithm. Sections 7.1 to 7.9 explain each of them using the same five points: what it is, why it is used in our project, how it works, how it contributes to the algorithm and what happens when it is combined with the others.

Table 7: The nine components of the hybrid retrieval algorithm

No.	Component	Library / tool	Role in the algorithm
7.1	TF-IDF word retrieval	scikit-learn	Scores how many important words match (score S_{word})
7.2	Character N-Gram retrieval	scikit-learn	Scores matching groups of 3 to 5 letters; handles spelling mistakes (S_{char})
7.3	MiniLM sentence embeddings	sentence-transformers	Turns each sentence into 384 numbers that represent its meaning
7.4	FAISS vector search	FAISS	Quickly finds the stored questions nearest in meaning (S_{sem})
7.5	Cosine similarity	scikit-learn / FAISS	Measure that converts two vectors into a score between 0 and 1
7.6	Hybrid score combination	Python	Adds the three scores; picks the highest combined score
7.7	Confidence threshold (0.35)	Python	Decides whether the best match is good enough to answer
7.8	Answer retrieval from CSV	pandas	Reads the answer stored for the best-matching question
7.9	Rejection of unrelated queries	Python	Shows "not found" when the best score is below 0.35

7.1 TF-IDF word-based retrieval

What it is. TF-IDF (Term Frequency – Inverse Document Frequency) converts a sentence into numbers by checking how important each word is. A word that occurs often in a question gets a higher weight (TF), and a word that occurs in very many stored questions gets a lower weight (IDF). Common words such as "is" and "the" therefore get a low score, and special words such as "attendance" or "CGPA" get a high score.

Scanned with OKEN Scanner

Why we use it. Most students use the same key words as the stored question ("minimum", "attendance", "exam"). Word matching is precise in this case and it clearly separates a question about attendance from a question about the library.

How it works. When the notebook starts, the vocabulary and the IDF values are learned from the 5,000 cleaned stored questions, and every stored question becomes one row of a sparse number matrix. For a new question, the same vocabulary is used to build one vector. The cosine similarity (Section 7.5) between the question vector and each stored-question vector is the word score S_{word}. A word of the question that is not in the vocabulary, for example a wrongly spelt word, gets no weight.

Contribution. It gives strong evidence when the wording is the same. In our test it alone reached 76.9% Top-1 accuracy, which is useful but not enough, because a spelling mistake or a changed word breaks the match.

In combination. Its score is one of the three terms of the combined score. When the student uses the stored words, the word score and the letter score are both high and reinforce each other. When a word is misspelt, the word score falls, and the other two scores must take over. This is the main reason why word matching is not used alone.

7.2 Character N-Gram retrieval

What it is. A character N-Gram is a small group of N letters taken from the text. In this project groups of 3 to 5 letters are used. The same TF-IDF idea is then applied to these letter groups instead of whole words. In the report this is called "letter matching".

Why we use it. Students make spelling mistakes. A misspelt word is a completely different word for word matching, but most of its letter groups are still the same as the correct word.

How it works. Each cleaned question is cut into overlapping groups of letters, and each group receives a TF-IDF weight. A question becomes a vector over letter groups, and the cosine similarity with every stored question gives the letter score S_{char}. Table 8 shows the idea with the wrongly spelt word "attendence" (an illustration of the technique, not a measurement from our experiments).

Table 8: Why letter groups survive a spelling mistake (illustration of the technique)

Representation	"attendance"	"attendence" (misspelt)	Shared
Whole word	attendance	attendence	none
3-letter groups	att, tte, ten, end, nda, dan, anc, nce	att, tte, ten, end, nde, den, enc, nce	5 of 8

As a whole word the misspelt word has nothing in common with the correct word, but 5 of its 8 three-letter groups are shared, so the two words still look close to each other. The same holds for 4-letter and 5-letter groups of the neighbouring letters.

Contribution. It makes the algorithm robust to spelling mistakes and small changes of words. In our test letter matching alone reached 97.2% Top-1 and 98.1% Top-3 accuracy, the best result of any single method, which is why it is given greater importance in the combined score.

In combination. It covers the weakness of word matching. Together with the word score it handles both correct and misspelt words. It still compares only the letters on the surface, so it cannot know that two differently worded sentences mean the same. This is the job of the third component.

7.3 MiniLM sentence embeddings

What it is. MiniLM is a small Transformer model (used through the sentence-transformers library) that has already learned the meaning of sentences. A sentence embedding is the list of 384 numbers that this model produces for a sentence. Sentences with the same meaning get similar numbers, even if the words are different. For example, "hall ticket issue date" and "when is the hall ticket given?" are close in meaning.

Scanned with OKEN Scanner

Why we use it. Students often ask the same thing in different words, for example "subject" instead of "course". Word and letter matching only compare symbols. The embedding compares meaning, so it helps to find questions that mean the same but are worded differently.

How it works. Each stored question is converted once into a 384-number vector when the index is built. When a student asks a question, the same model converts it into a vector at run time. Inside the model, attention (Section 1.3) lets every word look at all the other words of the sentence, and the result is a single vector for the whole sentence. We do not train the model; we only use it. It must be downloaded on the first run, which is why the first run takes a little longer.

Contribution. It adds meaning-based evidence, which the other two methods cannot provide.

In combination. Its vectors are not compared one by one by us. They are stored in FAISS (Section 7.4), which returns the semantic score S_{sem}. This score is added to the word and letter scores. The report does not give a separate accuracy for MiniLM alone, so its benefit is explained by design and is not claimed as a separate measured number.

7.4 FAISS vector similarity search

What it is. FAISS is a library from Facebook (Meta) that finds the nearest vectors among a large number of vectors very quickly.

Why we use it. The 5,000 stored questions are each a vector of 384 numbers. FAISS stores them in one index and searches this index efficiently. The same code would still work if the CSV grows to many more rows.

How it works. During index building, all question embeddings are added to a FAISS index. For a student question, its embedding is given to the index, and FAISS returns the stored questions whose vectors are nearest, together with their similarity values. When vectors are scaled to unit length, the inner product of two vectors is equal to their cosine similarity, which is the usual way of getting a cosine score from FAISS. The positions returned by FAISS are the row numbers of the CSV.

Contribution. It supplies the semantic score S_{sem} for the stored questions.

In combination. Because the word matrix, the letter matrix and the FAISS index are all built from the same CSV rows in the same order, row number i means the same stored question in all three. This lets the three scores be added row by row without any mapping.

7.5 Cosine similarity

What it is. Cosine similarity measures how close the directions of two vectors are. The result is a number: near 1 means very similar and near 0 means not related.

Why we use it. The same measure can be used for all three methods, so the three scores have the same meaning and can be added. It depends on the direction of the vectors and not on their length, so a short question and a long question about the same thing can still match.

How it works. For two vectors a and b, the dot product a·b is divided by the product of their lengths. The formula is given in Section 8.F, Equation (3). TF-IDF vectors contain no negative numbers, so their cosine lies between 0 and 1.

Contribution. It is the common score used by all three retrieval methods.

In combination. Because every method gives a score on the same 0 to 1 scale, the combined score is also on this scale, and one fixed threshold (0.35) can be applied to it.

7.6 Hybrid retrieval and score combination

What it is. Each of the three methods gives its own score for every stored question. The scores are added together, giving more importance to letter matching (because students make spelling mistakes) and some importance to meaning matching. The stored question with the highest combined score is the best match.

Scanned with OKEN Scanner

Why we use it. The methods fail in different situations. Word matching fails on spelling mistakes, letter matching cannot see meaning, and meaning matching alone can be confused by two sentences with similar topic. Their errors do not coincide, so adding the scores gives a more reliable ranking than any single score.

How it works. For each stored question i the combined score is $S_{final}(i) = w_{word} \cdot S_{word}(i) + w_{char} \cdot S_{char}(i) + w_{sem} \cdot S_{sem}(i)$, where the three weights are the fixed numbers set in the notebook. The report states only which method gets more importance; the numerical weights are defined in the notebook RAG_College_Chatbot.ipynb and are not repeated here, so that no value is assumed.

Contribution. It is the step that turns three separate searches into one decision. In the small sample of the original report (Table 10), the word score of the correct question is only 0.45 because of the spelling mistake, while the letter score is 0.82, and the question still gets the highest final score.

In combination. This is where all components meet: the three scores of Sections 7.1 to 7.4, on the common scale of Section 7.5, produce one final score per question, which goes to the threshold of Section 7.7.

7.7 Confidence threshold (0.35)

What it is. The threshold is a fixed limit on the best combined score. If the best score is 0.35 or more, the chatbot shows the answer. If it is lower, the chatbot says that it could not find the answer and asks the student to contact the Academic Office.

Why we use it. Every question always has some "best" stored question, even if it is unrelated. Without a limit the chatbot would answer a question such as "what is the weather today?" with some college rule. The threshold stops the chatbot from guessing.

How it works. The value was checked experimentally (Table 18). With a limit of 0.2 only 40.0% of the unrelated questions were refused, with 0.25 and 0.3 only 73.3%, and from 0.35 onwards 100.0%, while 100.0% of the college questions were still answered. 0.35 is the lowest tested limit at which both values are 100.0%. In the 18 test cases (Table 20), the lowest accepted score was 0.54 and the highest rejected score was 0.30.

Contribution. It makes the chatbot safe: it prefers to say "not found" than to give a wrong answer.

In combination. It acts on the combined score. A threshold on a single score would be less reliable, because for example a misspelt question has a low word score but should still be answered.

7.8 Final answer retrieval from the CSV

What it is. After the best match is found, the answer is read from the `answer` column of the CSV, in the same row as the best-matching question.

Why we use it. The answer is not generated by a model; it is the stored text written for the college rules. This is why the chatbot does not hallucinate, and why the rules can be updated by editing the CSV, with no training.

How it works. The search function returns the row numbers, the scores and the confidence. If the confidence is at least 0.35, the chatbot returns the answer of the first row number together with the matched question and the confidence score.

Contribution. It is the "retrieval" result of RAG that the student finally sees.

In combination. Its correctness depends completely on the earlier steps. If the wrong row is selected, a correct-looking but wrong answer is shown, so the quality of the hybrid score is what matters.

7.9 Rejection of unrelated or low-confidence queries

What it is. The behaviour of the chatbot when the best combined score is below 0.35. It shows: "Sorry, I could not find this in the college regulations or syllabus. Please contact the Academic Office."

Scanned with OKEN Scanner

Why we use it. The chatbot should answer only what is in the knowledge base. Questions like "what is the capital of France" or "how to cook rice" are not about college.

How it works. An unrelated question has few words, letter groups or meanings in common with the stored questions, so all three scores and therefore the combined score are low. In the reported test cases, the scores of such questions were 0.30, 0.26 and 0.22, and the weather question in the sample output had 0.25, all below 0.35.

Contribution. In the reported test, all 60 unrelated questions were refused, and all 300 college questions were answered.

In combination. The rejection works because all three methods agree that nothing is close. It protects against unrelated questions, but it cannot detect a wrong match with a high score. Test case 8 ("topics in unit 2 of DBMS") got a score of 0.67 for another subject, and is the one wrong case in the 18 test cases. Such cases are discussed in Chapter 13.

Scanned with OKEN Scanner

8. PROPOSED HYBRID RETRIEVAL ALGORITHM

> Algorithm name
>
> Hybrid Retrieval-Based RAG Algorithm using TF-IDF, Character N-Grams and MiniLM Sentence Embeddings
>
> The algorithm is the central technical contribution of this project. Everything else in the chatbot (the CSV, the notebook and the interface) exists to feed this algorithm or to show its result.

A. Algorithm overview

The algorithm works in two phases. Phase 1 (offline index preparation) runs once when the notebook starts: it loads the CSV, cleans the stored questions and builds three representations of them: a TF-IDF word matrix, a character N-Gram matrix and MiniLM embeddings stored in a FAISS index. Phase 2 (online answering) runs for every question typed by the student: the question is cleaned, scored by the three methods against all 5,000 stored questions, the three scores are combined, the best match is found, and its answer is shown only if its score is at least 0.35.

B. Objective of the algorithm

- To find the stored question that has the same meaning as the student's question, even when the question has spelling mistakes or different wording.
- To use the strengths of word, letter and meaning matching together instead of depending on one of them.
- To return only answers that are stored in the CSV, so that nothing is invented.
- To reject questions that are unrelated or not matched with enough confidence.
- To stay fast (about 3 milliseconds per question) and easy to update (edit the CSV).

C. Input and output

	Input	Output
Phase 1	The CSV file college_rag_qa_5000.csv with 5,000 rows (columns id, category, fact_id, question, answer)	Word matrix, character matrix and FAISS index built over the cleaned stored questions
Phase 2	One question typed by the student (spelling mistakes allowed)	If the confidence is at least 0.35: the stored answer, the matched question and the confidence score. Otherwise: the "not found" message

Scanned with OKEN Scanner

D. Technologies and techniques used

Table 9: Techniques used in the algorithm

Technique	Tool	Settings given in the report	Output
Text cleaning	Python	Lower case; remove filler words and special symbols	Cleaned text
TF-IDF word matching	scikit-learn	Whole words	Score S_{word}
Character N-Gram matching	scikit-learn	Groups of 3 to 5 letters	Score S_{char}
Sentence embeddings	sentence-transformers (MiniLM)	384 numbers per sentence	Vector for each question
Vector search	FAISS	Index over all question vectors	Score S_{sem}
Similarity measure	Cosine similarity	Score between 0 and 1	Similarity of two vectors
Score combination	Python	Weights fixed in the notebook; greater importance to letter matching	S_{final}
Confidence check	Python	Threshold 0.35	Answer or "not found"
Answer lookup	pandas	answer column of the best row	Answer text

E. Detailed working procedure

The 16 steps of the algorithm are grouped into the two phases. The numbers are the same as the numbered circles in the flowchart (Figure 3).

Phase 1: Offline index preparation

1. Load the CSV.The file college_rag_qa_5000.csv is read with pandas into a table of 5,000 rows, and the number of rows is checked.
2. Clean the stored questions. Each stored question is made lower case, and filler words and special symbols are removed.
3. Prepare the TF-IDF word representation. The vocabulary and IDF values are learned from the cleaned questions, and each question becomes a word-weight vector.
4. Prepare the character N-Gram representation. The same is done with letter groups of 3 to 5 letters.
5. Generate MiniLM embeddings. Each cleaned question is converted by MiniLM into a vector of 384 numbers.
6. Index with FAISS.All embeddings are added to a FAISS index so that the nearest vectors can be found quickly.

Phase 2: Online answering of a question

7. Accept the question. The student types a question in the notebook.
8. Clean the question. The same cleaning as in step 2 is applied, so that both texts have the same format.
9. Word-based retrieval. The question is converted into a word vector and its cosine similarity with all stored questions gives S_{word}.
10. Character-based retrieval. The question is converted into a letter-group vector and the cosine similarity gives S_{har}.
11. Semantic retrieval. The question is converted by MiniLM into a vector and FAISS finds the nearest stored questions, giving S_{sem}.

Scanned with OKEN Scanner

12. Combine the scores. For every stored question the three scores are combined into S_{final} using the weights set in the notebook.
13. Find the best match. The stored question with the highest S_{final} is selected, and this highest value is the confidence score. The question numbers are sorted, so that the first three give the Top-3 results.
14. Check the threshold. The confidence score is compared with 0.35.
15. If the confidence is at least 0.35, the answer in the same row of the CSV is retrieved and displayed with the matched question and the confidence.
16. Otherwise, the message "Sorry, I could not find this in the college regulations or syllabus. Please contact the Academic Office." is displayed.

F. Mathematical explanation of the retrieval techniques

Let N = 5,000 be the number of stored questions, c_i the cleaned stored question i, and q the cleaned user question. Only the formulas that are needed to explain the techniques are given. The report does not state any other formula, so none is added.

Text cleaning

$$q = \text{clean}(\text{query}), \quad c_i = \text{clean}(\text{question}_i), \quad i = 1, 2, \ldots, N \tag{1}$$

TF-IDF (word and letter versions)

The weight of a term t (a word, or a group of letters) in a text d is its term frequency multiplied by its inverse document frequency. The scikit-learn implementation uses a smoothed form of this and, by default, scales every vector to length 1.

$$w(t, d) = tf(t, d) \times idf(t), \quad idf(t) = \ln(N / df(t)) \tag{2}$$

Here tf(t, d) is how many times t occurs in d, and df(t) is the number of stored questions that contain t. A term that is present in many questions gets a small idf. For the letter version, t is a group of n letters with n = 3, 4, 5.

Cosine similarity

$$\cos(a, b) = (a \cdot b) / (|a| \times |b|) = \Sigma_k a_k b_k / (\sqrt{(\Sigma_k a_k^2)} \times \sqrt{(\Sigma_k b_k^2)}) \tag{3}$$

The three retrieval scores

$$S_{word}(i) = \cos(v_q^{word}, v_i^{word}) \tag{4}$$

$$S_{char}(i) = \cos(v_q^{char}, v_i^{char}) \tag{5}$$

$$S_{sem}(i) = \cos(e_q, e_i), \quad e = \text{MiniLM}(\text{text}) \in R^{384} \tag{6}$$

Here v are TF-IDF vectors and e are the 384-number MiniLM embeddings. FAISS returns the stored vectors that have the largest similarity with e_q. When the embeddings have length 1, the inner product $e_q \cdot e_i$ is equal to the cosine in Equation (3).

Hybrid score, best match and decision

$$S_{final}(i) = w_{word} S_{word}(i) + w_{char} S_{char}(i) + w_{sem} S_{sem}(i) \tag{7}$$

$$i^* = \text{argmax}_i S_{final}(i), \quad \text{confidence} = S_{final}(i^*) \tag{8}$$

$$\text{answer}(q) = \text{answer}_{best} \text{ if confidence} \geq 0.35; \quad \text{otherwise "not found"} \tag{9}$$

The weights w_{word}, w_{char} and w_{sem} are the fixed values set in the notebook. In the report, greater importance is given to letter matching and some importance to meaning matching. Figure 2 shows the combination.

Scanned with OKEN Scanner

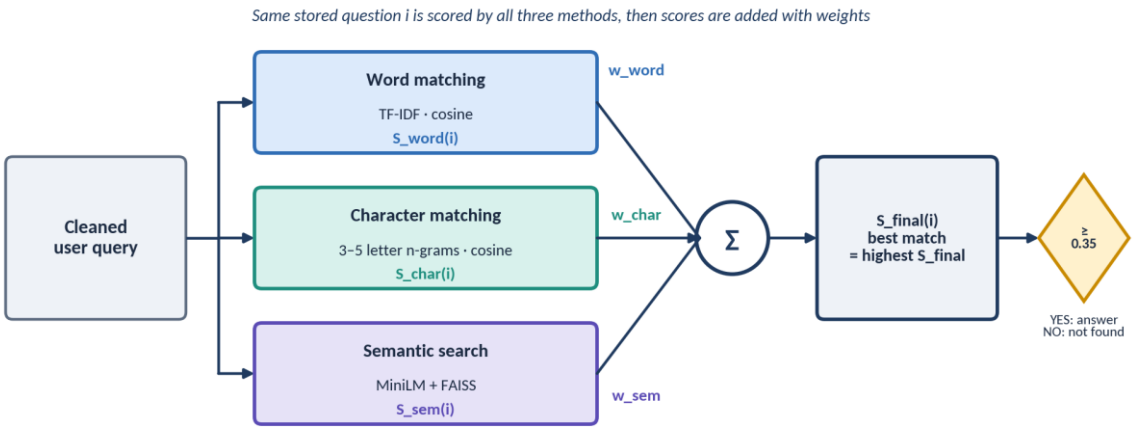


Figure 2: Combination of the three retrieval scores into one final score and the threshold decision

Table 10 reproduces the small example of the original report for the question "minimum attendence for exam" (with a spelling mistake). The values are sample values used for explanation in the report. They are not weights, and the table is not used to calculate the weights.

Table 10: Example of scores (sample values for explanation, from the original report)

Stored question	Word score	Letter score	Final score
What is the minimum attendance required for exams?	0.45	0.82	0.69
How is CGPA calculated?	0.00	0.08	0.06
What is the library fine for late books?	0.00	0.05	0.04

The first question gets the highest score, because many words and letters match, even though "attendence" is spelt wrongly. The word score alone (0.45) is low because of the spelling mistake, but the letter score (0.82) is high. This is why both are used.

Scanned with OKEN Scanner

G. Pseudocode

The pseudocode follows the structure of the notebook: building the indexes (class HybridRAG , created with use_dense=True), and the answer function that calls search and applies the check conf < 0.35 . Algorithm 1 is Phase 1 and Algorithm 2 is Phase 2.

Algorithm 1: BUILD_HYBRID_INDEX (Phase 1 – offline index preparation)

```
INPUT : CSV file "college_rag_qa_5000.csv" (id, category, fact_id, question, answer)
OUTPUT: word index, character index, FAISS semantic index, answer table

Initialization
  SET threshold ← 0.35
  SET weights w_word, w_char, w_sem ← values fixed in the notebook        // letter matching gets greater importance
  SET n-gram range ← 3 to 5 letters
  SET model ← pre-trained MiniLM sentence-embedding model                 // 384 numbers per sentence

Dataset loading
  READ the CSV into table df                                              // 5,000 rows

Preprocessing
  FOR EACH row i IN df DO
      clean_q[i] ← lower-case(df.question[i]); remove filler words and special symbols
  END FOR

Retrieval preparation
  M_word ← TF-IDF matrix of clean_q using whole words
  M_char ← TF-IDF matrix of clean_q using groups of 3 to 5 letters
  IF use_dense THEN
      E ← model.encode(clean_q)                                           // one 384-number vector per stored question
      faiss_index ← new FAISS index for 384-number vectors;               ADD E to faiss_index
  END IF
  RETURN M_word, M_char, faiss_index, df
```

Algorithm 2: ANSWER_QUESTION(query) (Phase 2 – online answering)

```
INPUT : user question "query"; M_word, M_char, faiss_index, df from Algorithm 1
OUTPUT: answer, matched question and confidence                           OR the "not found" message

Query processing
  q ← lower-case(query); remove filler words and special symbols          // same cleaning as Algorithm 1

Similarity calculation
  S_word ← cosine similarity of TF-IDF word vector of q with every row of M_word
  S_char ← cosine similarity of TF-IDF letter-group vector of q with every row of M_char
  IF use_dense THEN
      S_sem ← similarity of model.encode(q) with every stored vector, searched by FAISS
  ELSE S_sem ← 0        // meaning matching is skipped

Score combination
  FOR EACH stored question i DO
      S_final[i] ← w_word × S_word[i] + w_char × S_char[i] + w_sem × S_sem[i]
  END FOR
  SORT question numbers by S_final, highest first → ids                   // first 3 ids give Top-3
  best ← ids[0];  conf ← S_final[best]

Threshold checking
  IF conf < 0.35 THEN
      RETURN "Sorry, I could not find this in the college regulations or syllabus. Please contact
      the Academic Office."
  END IF

Final answer retrieval
  RETURN df.answer[best], df.question[best], conf
```

The weights are written as symbols because the report gives only their relative importance. Replace them by the notebook values when explaining the code.

Scanned with OKEN Scanner

H. Step-by-step execution

Figure 3 (below) shows the complete algorithm. The three retrieval methods have their own colour (blue: word, green: letter, violet: meaning), but they all start from the same cleaned question and join in the same score combination. Table 11 follows the same 16 steps.

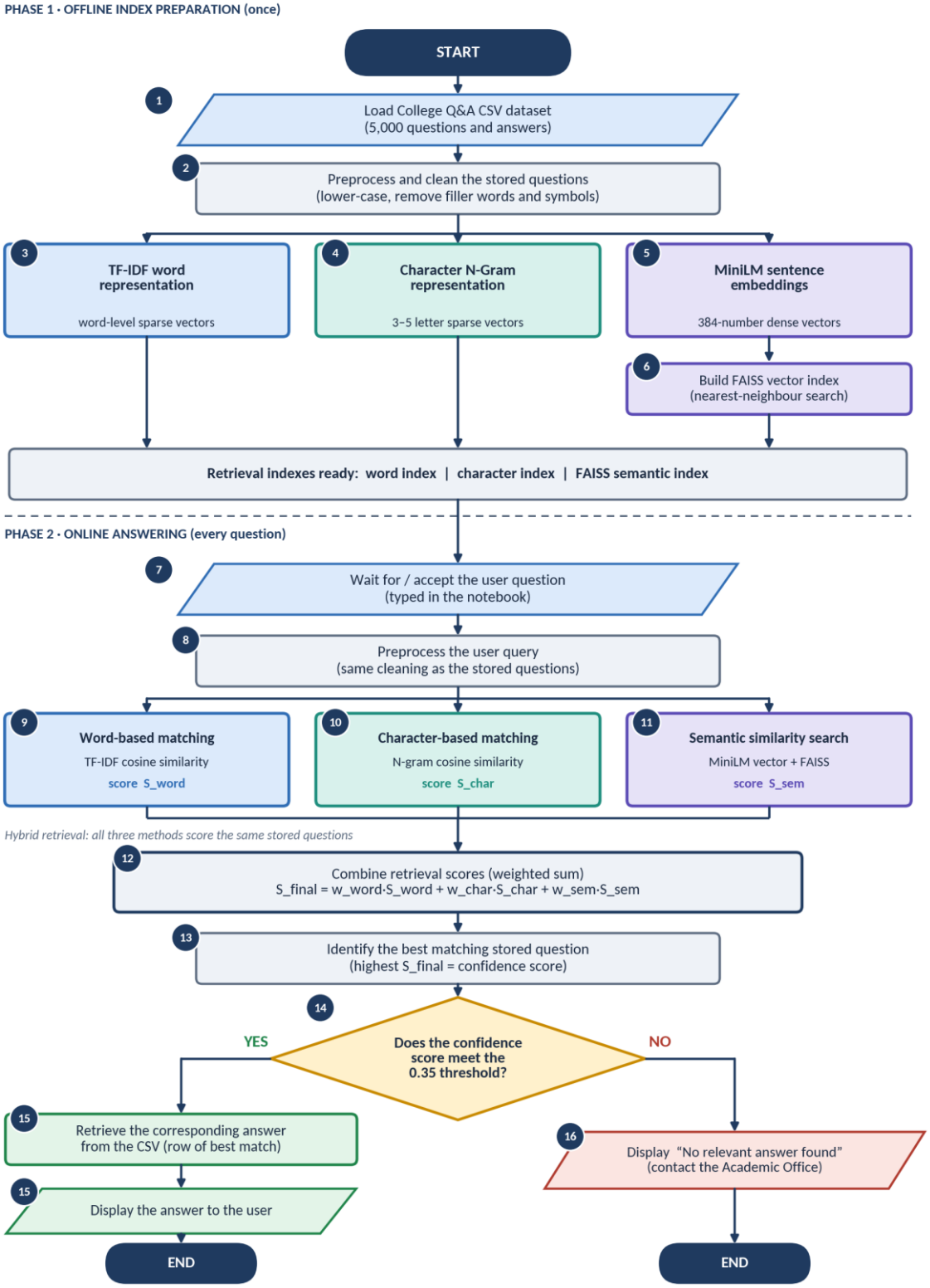


Figure 3: Flowchart of the proposed hybrid retrieval algorithm (numbered circles are the steps 1–16 of Section 8.E)

Scanned with OKEN Scanner

Table 11: Step-by-step execution of the algorithm

Step	Action	Technique	What is produced
1	Load CSV	pandas	Table of 5,000 questions and answers
2	Clean stored questions	Python	Cleaned, lower-case questions
3	Word representation	TF-IDF	Sparse word-weight vector for each question
4	Letter representation	Character N-Grams (3–5)	Sparse letter-group vector for each question
5	Embeddings	MiniLM	384-number vector for each question
6	Vector index	FAISS	Index for fast nearest-vector search
7	Accept question	Input	Text typed by the student
8	Clean question	Python	Cleaned query q
9	Word retrieval	TF-IDF + cosine	S_{word} for all stored questions
10	Letter retrieval	N-Grams + cosine	S_{char} for all stored questions
11	Semantic retrieval	MiniLM + FAISS	S_{sem} for the nearest stored questions
12	Combine scores	Weighted sum	S_{final} for each stored question
13	Best match	Highest S_{final}	Best question number and confidence
14	Threshold check	Confidence ≥ 0.35 ?	YES or NO
15	YES: answer	CSV lookup	Stored answer shown to the user
16	NO: not found	Message	"Sorry, I could not find this …" message

I. Worked example

The question "I have 72% attendance, can I write the exam?" and the result shown below are taken from the example in the original report (Table 12). The report gives the best match, its score and the answer, but it does not give the three separate scores S_{word}, S_{char} and S_{sem} of this question, so none is stated here.

Table 12: Example from input to output (from the original report)

Stage	What happens
Input	I have 72% attendance, can I write the exam?
Cleaning	i have 72% attendance can i write the exam
Search	Compared with all 5,000 stored questions
Best match	"I have 72% attendance, can I write the semester exam?" (score 0.88)
Check	0.88 is more than 0.35, so the answer is accepted
Output	With 72% attendance: You can apply for condonation with valid proof and a fee.

1. The system accepts the question typed by the student.
2. It preprocesses it: the text becomes lower case and the comma and the question mark are removed, and the words, the number 72 and the % sign remain.
3. It searches with the three techniques. Word matching finds the stored questions that share words such as "72%", "attendance", "write" and "exam". Letter matching compares the groups of letters of the same text, and would also work if a word was spelt wrongly. Meaning matching compares the vector of the question with the stored vectors in FAISS and finds questions about writing the exam with low attendance.
4. It combines the scores. Each stored question gets one final score S_{final} from its three scores.

Scanned with OKEN Scanner

5. It selects the best matching stored question, "I have 72% attendance, can I write the semester exam?". The only difference from the typed question is the word "semester".
6. It checks the threshold. The reported confidence is 0.88, which is higher than 0.35, so the answer is accepted.
7. It retrieves the CSV answer: "With 72% attendance: You can apply for condonation with valid proof and a fee."

In the sample output of the chatbot (Chapter 11), a slightly different typing of the same question, "I have 72% attendance can i write exam", gave the same answer with a confidence of 0.87. The report also notes that the remaining mistakes happen mostly when two questions are almost the same, for example the same question with 72% and 75% attendance. In this example the number 72 appears in both the typed question and the selected stored question.

J. Advantages of the selected algorithm

- Accurate with spelling mistakes and changed words: 97.6% Top-1 and 98.4% Top-3 accuracy on 800 test questions (reported).
- Uses three kinds of evidence: exact words, letter groups and meaning. A failure of one method can be covered by the others.
- No made-up answers: answers come only from the knowledge base. The chatbot refused all 60 unrelated test questions with the threshold 0.35.
- Very fast: about 3 milliseconds per question, so the chatbot replies instantly.
- Easy to update: change the CSV file and run again. No training is needed.
- Runs on free tools: Google Colab, scikit-learn, sentence-transformers and FAISS. No paid software.
- Easy to explain: each score is a cosine similarity, and each step has a simple meaning, so the algorithm can be explained step by step.

K. Why this algorithm was selected over simpler retrieval approaches

Word matching alone is not sufficient

Word matching treats "attendence" and "attendance" as two unrelated words, and treats "subject" and "course" as unrelated words. The student's question then loses its most important words. In our test with spelling mistakes and replaced words, word matching alone gave only 76.9% Top-1 and 81.4% Top-3 accuracy.

Character N-Grams handle spelling variations

Letter groups of 3 to 5 letters remain the same for most of a wrongly spelt word (Table 8). Letter matching alone raised the Top-1 accuracy from 76.9% to 97.2%, an increase of 20.3 percentage points, and the Top-3 accuracy to 98.1%.

Semantic embeddings find the same meaning in different words

Two questions can mean the same and still share few words or letters, for example "hall ticket issue date" and "when is the hall ticket given?". The MiniLM embedding gives similar numbers to such sentences, and FAISS finds them quickly. This covers the case that word and letter matching cannot cover.

Combining the techniques

The combined method gave 97.6% Top-1 and 98.4% Top-3 accuracy, compared with 76.9% and 81.4% for word matching and 97.2% and 98.1% for letter matching, at the same speed of about 3.3 ms per question. The reported improvement over letter matching alone is small (0.4 percentage points in Top-1 and 0.3 in

Scanned with OKEN Scanner

Top-3), because the 800 test questions were made by spelling mistakes and word replacement, which letter matching handles very well. The improvement over word matching is large (20.7 and 17.0 percentage points). The reported results do not show the contribution of MiniLM separately, so its inclusion is justified by design (different wording with the same meaning) and not by a separately measured number.

Table 13: Comparison of simpler retrieval approaches with the hybrid algorithm

Approach	Weakness for our problem	Reported Top-1 / Top-3	Time
Word matching only	Spelling mistakes and changed words break the match	76.9% / 81.4%	3.3 ms
Letter matching only	Compares only letters; cannot see that two sentences have the same meaning	97.2% / 98.1%	3.4 ms
Meaning matching only	Not reported separately in the report; meaning of similar topics can be confused	not reported	—
Hybrid (our algorithm)	Uses the three together; remaining errors when two questions are almost the same	97.6% / 98.4%	3.3 ms

9. System Architecture

The architecture shows how a question moves through the chatbot and how the answer is produced. The hybrid retriever (dashed box) is the core of the system; the other blocks prepare its input or present its output.

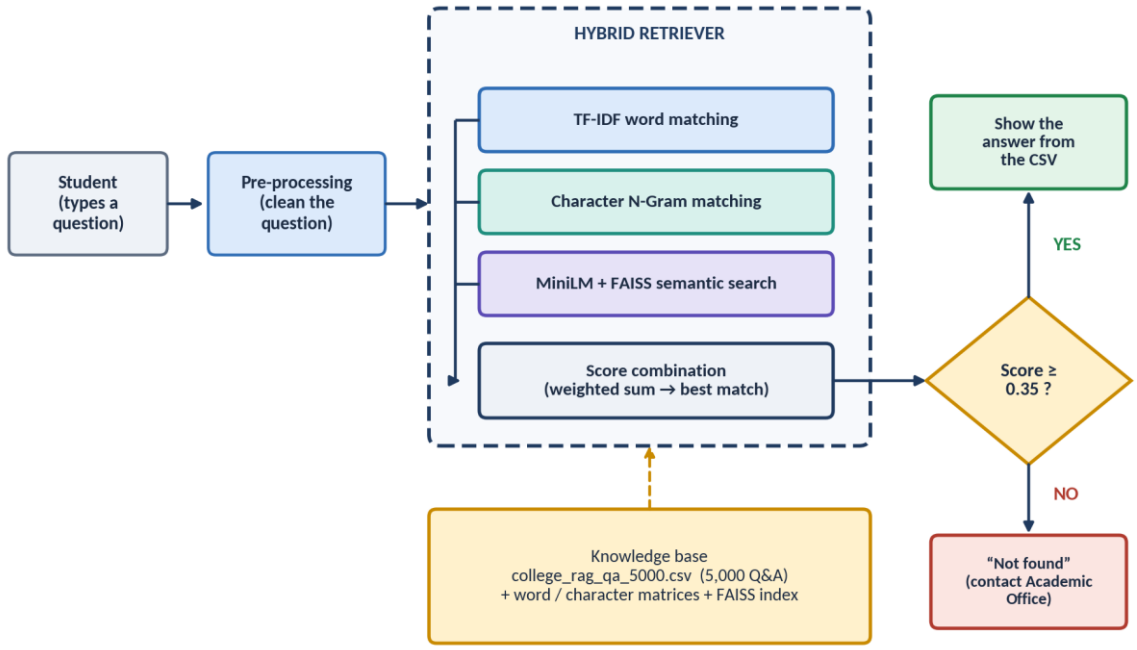


Figure 4: System architecture of the RAG chatbot with the hybrid retriever

9.1 Description of each part

Student: types the question in the notebook.

Pre-processing: converts the text to a standard format (lower case, no filler words or special symbols).

Knowledge base: the CSV file with 5,000 questions and answers. Together with the word matrix, the letter matrix and the FAISS index built from it, it is the only source of answers.

Hybrid retriever: compares the question with every stored question using word, letter and meaning matching (Sections 7.1 to 7.4).

Scanned with OKEN Scanner

Score combination: adds the three scores with the weights fixed in the notebook and picks the stored question with the highest score (Section 7.6).

Confidence check: shows the answer only if the score is 0.35 or more (Section 7.7).

Answer or not found: the final output given to the student: the stored answer with the matched question and confidence, or the "not found" message that asks the student to contact the Academic Office.

10. Implementation

The chatbot is implemented in Python in Google Colab. The notebook file is RAG_College_Chatbot.ipynb . The retrieval algorithm of Chapter 8 is implemented as the class HybridRAG .

10.1 Technology stack

Table 14: Tools used

Tool	Use
Python	Programming language
Google Colab	Free online notebook to run the code
pandas	Reading the CSV file
scikit-learn	TF-IDF word and letter matching
sentence-transformers (MiniLM)	Meaning matching
FAISS	Fast search of the nearest sentences
matplotlib	Charts

10.2 Steps of implementation

1. Install the libraries and upload the CSV file.
2. Load the CSV using pandas and check the number of rows.
3. Clean the questions and build the word, letter and meaning indexes.
4. Write the function that finds the best match and checks the score.
5. Ask questions and see the answers.
6. Test with 800 spelling-mistake questions and measure accuracy.
7. Run the interactive loop where anyone can type a question.

10.3 Important code

Loading the data:

```
df = pd.read_csv("college_rag_qa_5000.csv")
print("Rows:", len(df))
```

Asking a question (the chatbot returns the answer and the confidence):

```
rag = HybridRAG(df, use_dense=True)   # builds the indexes
ans = rag.answer("What is the minimum attendance for exams?")
print(ans["answer"], ans["confidence"])
```

Inside the answer function, the best match is found and the confidence check is applied:

```
ids, scores, conf = self.search(query)
if conf < 0.35:
    return "Sorry, I could not find this ..."
return self.df.answer[ids[0]]
```


Scanned with OKEN Scanner

The setting use_dense=True switches on the meaning matching (MiniLM and FAISS). The input and output of a complete example are given in Table 12 of Section 8.I.

10.4 How to run the notebook

1. Open Google Colab (colab.research.google.com) and upload RAG_College_Chatbot.ipynb .
2. Click Runtime, then Run all.
3. When asked, upload the file college_rag_qa_5000.csv .
4. Wait for the index to be built (about a minute for the first run).
5. Read the sample answers and the accuracy results shown by the notebook.
6. In the last cell, type your own question and press Enter. Type exit to stop.

To use the real college regulations, replace the CSV with a file that has the same columns (question and answer are required).

Files submitted with this report: RAG_College_Chatbot.ipynb (Google Colab code), college_rag_qa_5000.csv (dataset), PPT and A5 poster.

11. Results and Discussion

The chatbot was tested in Google Colab. The results are explained below in simple terms.

11.1 Nature of the evidence

All numerical results in this chapter were measured by the project team in its Colab experiments and are reproduced unchanged from the original report. For this revised report the notebook and dataset were not re-run, so the results have not been independently verified . Only simple consistency checks on the reported numbers were possible (for example, the category counts in Table 19 add up to 800 questions and 781 correct answers, which is 97.6%, the same as the combined Top-1 accuracy).

Table 15: Status of the reported results

Result	Status	Remark
Word-only accuracy: 76.9% Top-1, 81.4% Top-3	Reported by the team	800 test questions
Letter-only accuracy: 97.2% Top-1, 98.1% Top-3	Reported by the team	800 test questions
Hybrid accuracy: 97.6% Top-1, 98.4% Top-3	Reported by the team	800 test questions
Time per question: about 3 ms	Reported by the team	3.3 ms and 3.4 ms in the table
Unrelated refused 60 of 60; college answered 300 of 300	Reported by the team	Threshold 0.35; measured with word and letter matching
Category-wise accuracy	Reported; totals checked	Counts add up to 800; 781 correct = 97.6%
18 test cases: 17 correct	Reported by the team	Case 8 wrong
Dedicated MiniLM-only accuracy	Not reported	Results of meaning matching are shown in the notebook

11.2 Output of the chatbot

Table 16 shows questions typed by a user and the answers given by the chatbot with the confidence score.

Scanned with OKEN Scanner

Table 16: Sample questions and answers of the chatbot

Question typed	Answer given	Confidence
What is the minimum attendance for exams?	A student must have at least 75% overall attendance to be eligible for semester-end exams.	0.60
I have 72% attendance can i write exam	With 72% attendance: You can apply for condonation with valid proof and a fee.	0.87
which subject covers alpha-beta pruning	Alpha-beta pruning is covered in Unit 3 of Artificial Intelligence (25CS301).	1.00
convert 8.4 cgpa to percentage	A CGPA of 8.4 is equivalent to 76.5% using Percentage = (CGPA - 0.75) x 10.	0.95
What is the weather today?	Sorry, I could not find this in the college regulations or syllabus. Please contact the Academic Office.	0.25

11.3 How we tested

We took 800 questions from the dataset and changed them: some words were replaced with other words (for example subject to course) and some words were spelt wrongly. We hid the original question from the chatbot, so it had to find the answer from other ways of asking the same thing. If the chatbot returned the correct fact (checked with the `fact_id` column), it was counted as correct.

11.4 Accuracy and speed

Table 17: Accuracy on 800 test questions

Method	Top-1 accuracy	Top-3 accuracy	Time per question
Word matching only	76.9%	81.4%	3.3 ms
Letter matching only	97.2%	98.1%	3.4 ms
Combined (our method)	97.6%	98.4%	3.3 ms

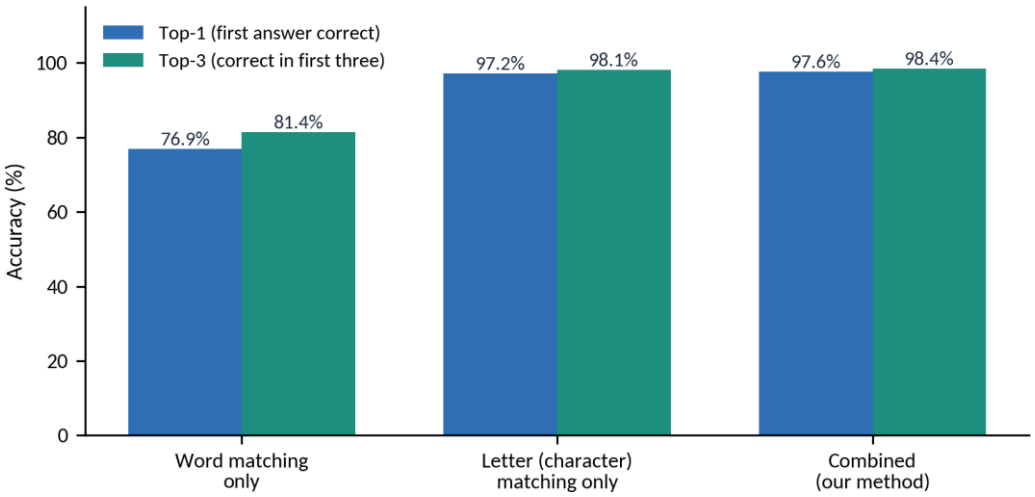


Figure 5: Accuracy of the three methods

Top-1 means that the first answer given was correct. Top-3 means that the correct answer was in the first three results.

11.5 Discussion

- Word-only retrieval (76.9% Top-1, 81.4% Top-3).A spelling mistake or a changed word breaks the match, so about one in four first answers is wrong.

Scanned with OKEN Scanner

- Character-only retrieval (97.2% Top-1, 98.1% Top-3). Most letters of a wrongly spelt word are still correct, so this single method already gives a very high accuracy on this test set.
- Hybrid retrieval (97.6% Top-1, 98.4% Top-3).Combining the methods gave the best result in both measures. The gain over letter matching alone is small on this test set, and the gain over word matching alone is large.
- Query processing time. Each answer takes only about 3 milliseconds (3.3 ms for word and for the combined method, 3.4 ms for letter matching), so the chatbot replies instantly.
- Remaining errors. The remaining mistakes happen mostly when two questions are almost the same, for example the same question with 72% and 75% attendance.

11.6 Questions that are not about college

We also tried 60 unrelated questions such as "what is the weather today?" and "tell me a joke". With the limit of 0.35, the chatbot refused all 60 and still answered all 300 college questions we tried. This shows that the chatbot does not guess. Table 18 and Figure 6 show the effect of the limit.

Table 18: Effect of the confidence limit

Limit (threshold)	Unrelated questions refused	College questions answered
0.2	40.0%	100.0%
0.25	73.3%	100.0%
0.3	73.3%	100.0%
0.35	100.0%	100.0%
0.4	100.0%	100.0%

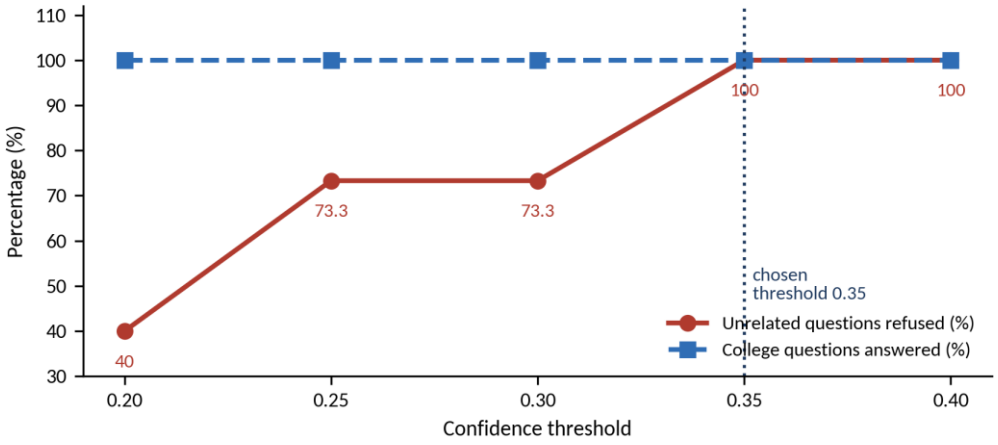


Figure 6: Effect of the confidence threshold on refusing unrelated questions and answering college questions (values of the table above)

Note: these numbers were measured with the word and letter matching. When the notebook is run in Google Colab, the meaning matching (MiniLM) is also added, and the results are shown in the notebook.

11.7 Accuracy for each category

The same 800 test questions are grouped by category in Table 19 and Figure 7. A correct result means the chatbot returned the right fact.

Scanned with OKEN Scanner

Table 19: Category-wise accuracy

Category	Test questions	Correct	Accuracy
Syllabus	548	529	96.5%
CGPA	70	70	100.0%
Attendance	58	58	100.0%
Academics	24	24	100.0%
Grading	22	22	100.0%
Exams	18	18	100.0%
Backlogs	13	13	100.0%
Placements	12	12	100.0%
Project	12	12	100.0%
Fees	11	11	100.0%
Hostel	6	6	100.0%
Library	6	6	100.0%

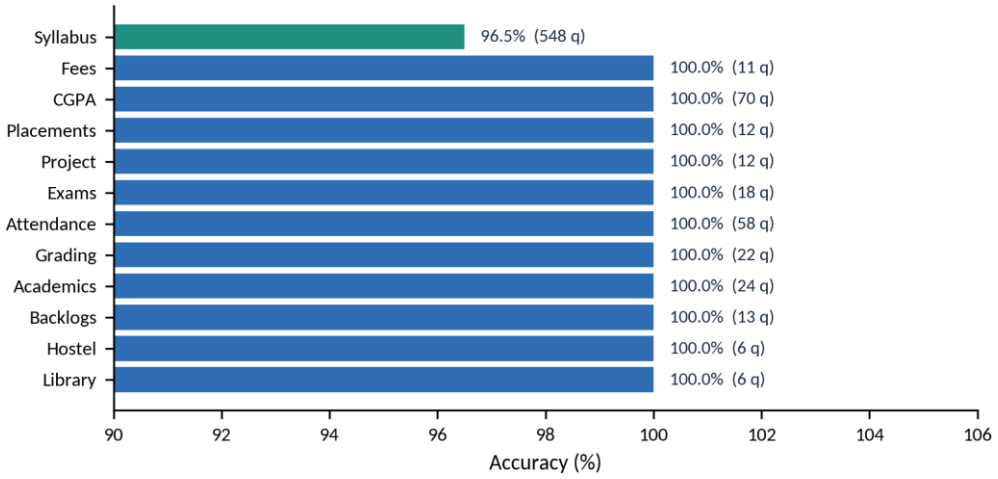


Figure 7: Accuracy for each category

Rules about attendance, CGPA, exams and fees were answered almost perfectly. The syllabus category has the most rows and the most similar questions (for example Unit 2 and Unit 3 of the same subject), so a few more mistakes happen there.

11.8 Test cases

We also tried 18 new questions typed by us, including spelling mistakes, short forms and unrelated questions (Table 20).

Table 20: Test cases

No.	Question typed	Answer given (short)	Score	Result
1	What is the minimum attendance required?	A student must have at least 75% overall attendance to be eligible for...	0.71	Correct
2	how many credits are needed for B.Tech	A B.Tech student must complete 160 credits as per the curriculum to be...	0.78	Correct
3	what are the grade ponits for A+	Grades and points: O=10, A+=9, A=8, B+=7, B=6, C=5, P=4, F=0 and Ab=0 ...	0.54	Correct
4	when are supplementry exams held	Supplementary exams are held once a year, usually along with the odd o...	0.65	Correct
5	what is the library fine for late books	The late fine is Rs. 2 per day per book; a lost book must be replaced ...	0.62	Correct

Scanned with OKEN Scanner

No.	Question typed	Answer given (short)	Score	Result
6	which textbook is used for Computer Networks	The prescribed textbook for Computer Networks is Behrouz Forouzan, Dat...	0.78	Correct
7	what is the course code of Machine Learning	The course code of Machine Learning is 25CS401.	1.00	Correct
8	topics in unit 2 of DBMS	Unit 2 of Software Engineering covers: Requirements engineering, SRS, ...	0.67	Wrong
9	eligibility for campus placements	Eligible students have CGPA >= 6.0, no active backlogs, and at least 7...	0.96	Correct
10	how do i apply for revaluation	Apply online within 7 days of result declaration, paying Rs. 1,000 per...	1.00	Correct
11	convert 7.6 cgpa to percentage	A CGPA of 7.6 is equivalent to 68.5% using Percentage = (CGPA - 0.75) ...	0.95	Correct
12	what is the team size for mini capstone	Each AI Mini-Capstone batch consists of 4 students, and all members mu...	0.90	Correct
13	hostel in time	Hostel in-time is 8:00 PM for first years and 9:00 PM for others; late...	0.71	Correct
14	which subject covers kmeans	K-means is covered in Unit 3 of Machine Learning (25CS401).	1.00	Correct
15	can i change my branch	Branch change is allowed at the end of the first year for students wit...	1.00	Correct
16	what is the capital of France	Sorry, I could not find this in the college regulations or syllabus. P...	0.30	Correct
17	who won the IPL	Sorry, I could not find this in the college regulations or syllabus. P...	0.26	Correct
18	how to cook rice	Sorry, I could not find this in the college regulations or syllabus. P...	0.22	Correct

17 out of 18 were correct. The wrong one is test case 8: the student wrote the short form "DBMS", which is not present in the dataset, so the chatbot matched another subject. The score of 0.67 was above the threshold, so the confidence check could not catch this error. The remedy is to add short forms such as DBMS, OS, CN and ML to the CSV, which is listed in the future scope. The three unrelated questions (cases 16 to 18) had scores of 0.30, 0.26 and 0.22, below 0.35, and were correctly refused.

11.9 What the confidence score means

Table 21: Meaning of the confidence score

Score	Meaning	What the chatbot does
0.80 to 1.00	Almost the same question is stored	Shows the answer
0.35 to 0.79	Similar question found (different wording or spelling mistakes)	Shows the answer
Below 0.35	No related question in the dataset	Says "not found" and asks the student to contact the Academic Office

12. Real-World Applications

- Student help desk: a 24x7 chatbot on the college website or WhatsApp.
- Admission and fee office: answers about eligibility, fees, scholarships and hostel.
- Examination cell: hall ticket, revaluation, supplementary exam and backlog rules.
- Teachers: quick look-up of syllabus, credits and exam pattern.
- Other colleges and schools: only the CSV file needs to be replaced.
- Companies: HR policy and customer support FAQ chatbots can use the same idea.

Scanned with OKEN Scanner

13. Limitations

- The dataset in this project is a sample. Real regulations of the college must be added before use.
- The chatbot can answer only what is in the CSV. It cannot combine many rules to think out a new answer.
- It works only in English.
- Each question is answered separately. The chatbot does not remember the earlier conversation.
- When two questions are almost the same, it may choose the wrong one. A wrong match with a score above 0.35 is not caught by the threshold (test case 8).
- Short forms such as DBMS or OS are understood only if they are present in the dataset.
- The meaning-matching model must be downloaded, so the first run takes some time.
- The reported threshold experiment was measured with word and letter matching; the effect of meaning matching on the threshold is shown in the notebook and is not repeated in this report.

14. Future Scope

- Read the official regulation PDF files directly, so that the CSV need not be written by hand.
- Use an LLM to write the final answer in a friendly way and show which rule it used.
- Add support for Telugu and Hindi, and voice input.
- Remember the previous questions in a conversation.
- Add short forms and nicknames of subjects to the dataset.
- Put the chatbot on the college website or WhatsApp.
- Give personal answers, such as the student's own attendance status, by connecting to the college database.

15. Conclusion

In this project, we built a RAG chatbot that answers college regulation and syllabus questions. Its heart is the hybrid retrieval algorithm: TF-IDF word matching, character N-Gram letter matching and MiniLM meaning matching with FAISS are used together on a CSV of 5,000 questions and answers, their cosine-similarity scores are combined, and the best match is shown only if its confidence is at least 0.35. Otherwise the chatbot says "not found".

In our tests, the chatbot gave the correct answer first time for 97.6% of the questions that had spelling mistakes and changed words (98.4% within the first three), against 76.9% for word matching alone, and refused all unrelated questions. It is fast, easy to update and does not give made-up answers. Through this project, we learned how LLMs, Transformers and RAG are used to solve a real problem, and the same idea can be used in any college or company.

16. References

[1] P. Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS, 2020.

[2] A. Vaswani et al., Attention Is All You Need, NeurIPS, 2017.

[3] N. Reimers and I. Gurevych, Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, EMNLP, 2019.

Scanned with OKEN Scanner

[4] J. Johnson, M. Douze and H. Jegou, Billion-scale similarity search with GPUs (FAISS), IEEE Transactions on Big Data, 2019.

[5] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, Pearson.

[6] scikit-learn documentation: https://scikit-learn.org

[7] sentence-transformers documentation: https://www.sbert.net

[8] Google Colab: https://colab.research.google.com

Scanned with OKEN Scanner

17. Appendix A: Source Code

This appendix reproduces the complete Python source of the Google Colab notebook RAG_College_Chatbot.ipynb, which implements the hybrid retrieval algorithm of Chapter 8 (class `HybridRetriever`) and the chatbot (class `CollegeChatbot`). Code cells are shown in the order in which they run. Long lines are wrapped for printing, and decorative emoji characters in strings are omitted. The regulations and syllabus in the CSV are sample data and must be replaced with official college documents before real use.

Section 2 - Install Dependencies

```
# Colab already has pandas/numpy/sklearn/matplotlib/seaborn. Install the rest quietly.
!pip -q install sentence-transformers faiss-cpu ipywidgets

import os, re, time, json, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import confusion_matrix
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

# Optional heavy libraries: the notebook degrades gracefully if they are missing.
try:
    import faiss; HAS_FAISS = True
except Exception: HAS_FAISS = False
try:
    from sentence_transformers import SentenceTransformer; HAS_ST = True
except Exception: HAS_ST = False
print("FAISS available:", HAS_FAISS, "| MiniLM available:", HAS_ST)

# ---------------- global configuration ----------------
CSV_PATH    = "college_rag_qa.csv"
INSTITUTION = "B.Tech Engineering College"   # main prototype focus
SEED        = 42
NOT_FOUND_MSG = "Sorry, this information is not available in the current college knowledge base.
    Please contact the Academic Office."
```

Section 3 - Dataset Loading

```
import glob
def load_dataset(path=CSV_PATH):
    # Find the CSV under ANY file name (e.g. 'Document from Susmitha vura'); otherwise ask for an
        upload.
    if not os.path.exists(path):
        cands = [f for f in glob.glob("*.csv") + glob.glob("/content/*.csv") if "rag_artifacts" not
            in f]
        if cands:
            path = max(cands, key=os.path.getsize); print("Using CSV found at:", path)
        else:
            try:
                from google.colab import files
                print("Please upload your college_rag_qa.csv file (any file name is fine)")
                up = files.upload(); path = list(up.keys())[0]
            except ImportError:
                raise FileNotFoundError("CSV not found. Put it in the same folder as this
                    notebook.")
    df = pd.read_csv(path)
    needed = {"fact_id","institution_type","category","semester","subject","unit","question","answer
        ","source_type","policy_status"}
    missing = needed - set(df.columns)
    if missing: raise ValueError(f"CSV is missing columns: {missing}. Did you upload the right file
        (not the .ipynb)?")
    if "question_style" not in df.columns: df["question_style"] = "unknown"
    return df
raw = load_dataset()
print("Dimensions (all institutions):", raw.shape)
kb = raw[raw.institution_type == INSTITUTION].reset_index(drop=True)
print("B.Tech knowledge base used by the chatbot:", kb.shape)
display(kb.head(10))
print("\nCategory distribution:"); display(kb.category.value_counts())
print("Missing values per column:"); display(raw.isna().sum())
print("Duplicate questions:", raw.question.str.lower().duplicated().sum(), "| duplicate Q-A pairs:",
     raw.duplicated(["question","answer"]).sum())
```


Scanned with OKEN Scanner

Section 4 - Exploratory Data Analysis

```
fig, ax = plt.subplots(2, 3, figsize=(18, 9))
order = kb.category.value_counts().index
sns.countplot(y="category", data=kb, order=order, ax=ax[0,0], palette="viridis");
    ax[0,0].set_title("Questions by category")
raw.institution_type.value_counts().plot.pie(autopct="%1.1f%%", ax=ax[0,1]); ax[0,1].set_ylabel("");
     ax[0,1].set_title("Records by institution type (non-B.Tech = labelled samples)")
sns.countplot(x="semester", data=kb, ax=ax[0,2], palette="mako"); ax[0,2].set_title("Semester-wise
    distribution (All = applies to every semester)")
kb["q_len"] = kb.question.str.split().str.len(); kb["a_len"] = kb.answer.str.split().str.len()
sns.histplot(kb.q_len, bins=range(1,25), ax=ax[1,0], color="teal"); ax[1,0].set_title("Question
    length (words)")
sns.histplot(kb.a_len, bins=30, ax=ax[1,1], color="coral"); ax[1,1].set_title("Answer length
    (words)")
sns.countplot(y="question_style", data=kb, ax=ax[1,2],
    order=kb.question_style.value_counts().index[:8]); ax[1,2].set_title("Question-writing styles
    (top 8)")
plt.tight_layout(); plt.show()
print("Unique answers (facts):", kb.fact_id.nunique(), "| avg. question variants per fact:",
    round(len(kb)/kb.fact_id.nunique(),1))
```

Section 5 - Text Preprocessing

```
ABBR = {"sem":"semester","sems":"semester","exams":"exam","examination":"exam","examinations":"exam"
    ,"min":"minimum","max":"maximum",
        "reval":"revaluation","supply":"supplementary","suply":"supplementary","laboratory":"lab","m
            aths":"mathematics","math":"mathematics",
        "phy":"physics","chem":"chemistry","dept":"department","cert":"certificate","info":"informat
            ion","u":"you","ur":"your","r":"are",
        "wat":"what","whats":"what is","subs":"subjects","subjects":"subject","att":"attendance","at
            tn":"attendance","regs":"regulations","pps":"programming for problem solving"}
FILLER_PHRASES = [r"\bi want to know\b", r"\bcan you tell me\b", r"\bquick question\b", r"\bplease
    tell( me)?\b", r"\bone doubt\b", r"\bi have a doubt\b", r"\bdoubt\b", r"\btell me\b"]
FILLER_WORDS  = {"please","pls","plz","sir","madam","bro","hi","hello","hey","kindly","asap"}

def clean_text(text: str) -> str:
    # Normalise a query/question for matching; keep numbers and technical terms.
    t = str(text).lower()
    t = t.replace("%", " percent ")
    t = re.sub(r"unit\s*(\d)", r"unit \1", t)            # unit1 -> unit 1
    for p in FILLER_PHRASES: t = re.sub(p, " ", t)
    t = re.sub(r"[^a-z0-9\s]", " ", t)                    # punctuation -> space
    toks = [ABBR.get(w, w) for w in t.split() if w not in FILLER_WORDS]
    out = " ".join(toks).strip()
    return out if out else str(text).lower().strip()

for q in ["Pls tell me min attendence for sem exams??", "I have 72% attendance, can I write exam",
    "unit1 of maths"]:
    print(q, "->", clean_text(q))
```


Scanned with OKEN Scanner

Section 6 - Hybrid Retrieval Algorithm

```
class HybridRetriever:
    # Hybrid TF-IDF + char n-gram + MiniLM/FAISS retriever over stored questions.
    def __init__(self, weights=(0.3, 0.4, 0.3),
        model_name="sentence-transformers/all-MiniLM-L6-v2"):
        self.weights, self.model_name = tuple(weights), model_name

    def fit(self, questions):
        self.clean_q = [clean_text(q) for q in questions]
        self.word_vec = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)
        self.char_vec = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), sublinear_tf=True)
        self.Xw = self.word_vec.fit_transform(self.clean_q)
        self.Xc = self.char_vec.fit_transform(self.clean_q)
        self.use_dense = HAS_ST
        if self.use_dense:
            try:
                self.encoder = SentenceTransformer(self.model_name)
                self.E = self._embed(self.clean_q)
                if HAS_FAISS:
                    self.index = faiss.IndexFlatIP(self.E.shape[1]); self.index.add(self.E)
            except Exception as ex:      # model download / memory problem -> keep working without
                MiniLM
                print("MiniLM could not be loaded (", type(ex).__name__, ") -> using word + char
                    scores only.")
                self.use_dense = False
        if not self.use_dense:
            print("MiniLM not available -> running with word + char scores only (weights
                renormalised).")
        return self

    def _embed(self, texts):
        return self.encoder.encode(list(texts), normalize_embeddings=True,
            show_progress_bar=False).astype("float32")

    def score_matrices(self, queries):
        # Return (S_word, S_char, S_sem), each of shape (n_queries, n_kb_questions) on a 0-1 cosine
            scale.
        cq = [clean_text(q) for q in queries]
        Sw = (self.word_vec.transform(cq) @ self.Xw.T).toarray()
        Sc = (self.char_vec.transform(cq) @ self.Xc.T).toarray()
        if self.use_dense:
            Eq = self._embed(cq); n = self.E.shape[0]
            if HAS_FAISS:
                D, I = self.index.search(Eq, n)                # FAISS vector search (all
                    neighbours)
                Ss = np.zeros_like(D); np.put_along_axis(Ss, I, D, axis=1)
            else:
                Ss = Eq @ self.E.T
            Ss = np.clip(Ss, 0, 1)
        else:
            Ss = np.zeros_like(Sw)
        return Sw, Sc, Ss

    def combine(self, Sw, Sc, Ss, weights=None):
        w = np.array(weights or self.weights, dtype=float)
        if not self.use_dense: w[2] = 0
        w = w / w.sum()                                        # weights always sum to 1 -> score
            stays in 0-1
        return w[0]*Sw + w[1]*Sc + w[2]*Ss

    def search(self, query, k=3, weights=None):
        S = self.combine(*self.score_matrices([query]), weights)[0]
        top = np.argsort(-S)[:k]
        return top, S[top]
```


Scanned with OKEN Scanner

Section 7 - Confidence and Relevance Checking

```
def split_by_fact(df, val_frac=0.15, test_frac=0.15, seed=SEED):
    rng = np.random.RandomState(seed); split = np.array(["train"] * len(df), dtype=object)
    for fid, idx in df.groupby("fact_id").indices.items():
        if len(idx) >= 8:
            idx = rng.permutation(idx); nv = max(1, int(len(idx)*val_frac)); nt = max(1,
                int(len(idx)*test_frac))
            split[idx[:nv]] = "val"; split[idx[nv:nv+nt]] = "test"
    return split
kb["split"] = split_by_fact(kb)
print(kb.split.value_counts().to_dict())

OOD = ["What is the weather today?","Who won the cricket match yesterday?","How do I bake a
    chocolate cake?","What is the capital of France?","Tell me a joke","Who is the prime minister of
     India?",
"Best smartphone under 20000","How to lose weight fast?","Latest movie releases this week","What is
    the price of petrol today?","Explain quantum computing in simple words","How to learn guitar?",
"Which is the tallest mountain in the world?","How do I book a train ticket?","What is bitcoin
    price?","Write a poem about love","How to make biryani?","Who is the richest person in the
    world?",
"What is the meaning of life?","Recommend a good web series","How to fix a flat bicycle tyre?","What
     is the stock price of Tesla?","Translate hello into Spanish","Who invented the telephone?",
"When is the next IPL match?","How far is the moon from earth?","Which actor is the best in
    Tollywood?","How to apply for a passport?","Tell me about the Mughal empire","What is the best
    programming language for AI jobs?",
"What is the mess menu on Sunday?","What was the placement package of CSE last year?","Which
    cafeteria item is cheapest?","When is the college annual day fest?","What is the principal's
    phone number?",
"Does the college have a swimming pool?","What is the cut-off rank for EAMCET admission?","How many
    seats are there in ECE?","Is there wifi in the hostel rooms?","What is the dress code for
    college fest?",
"What are the subjects in the fifth semester?","What is Unit 3 of Machine Learning?","What is the
    syllabus of Data Structures?","Explain the syllabus of Operating Systems unit 2","Who is the HOD
     of Mechanical department?",
"How to get a visa for studying abroad?","What is the GATE exam syllabus?","How to prepare for civil
     services?","Which company visits campus for placements?","Can I get a refund for my gym
    membership?",
"How do I reset my Instagram password?","What is the speed of light?","Describe photosynthesis","How
     do airplanes fly?","What is the population of India?","Give me a recipe for pasta","Who wrote
    Hamlet?","How to start a YouTube channel?","What is the best laptop for gaming?","Where is the
    nearest hospital?"]
rng = np.random.RandomState(SEED); perm = rng.permutation(len(OOD))
ood_val  = [OOD[i] for i in perm[: len(OOD)//2]]
ood_test = [OOD[i] for i in perm[len(OOD)//2 :]]
print("Unrelated queries -> val:", len(ood_val), "test:", len(ood_test))

train = kb[kb.split == "train"].reset_index(drop=True)
val   = kb[kb.split == "val"].reset_index(drop=True)
test  = kb[kb.split == "test"].reset_index(drop=True)
t0 = time.time(); eval_ret = HybridRetriever().fit(train.question.tolist()); print(f"Index built on
    {len(train)} questions in {time.time()-t0:.1f}s")

def rank_metrics(S, true_fids, fids):
    # Top-1 / Top-3 (distinct facts) accuracy + best confidence for each query.
    order = np.argsort(-S, axis=1)[:, :40]; top1, top3 = [], []
    for row, t in zip(order, true_fids):
        seen = []
        for j in row:
            if fids[j] not in seen: seen.append(fids[j])
            if len(seen) == 3: break
        top1.append(seen[0] == t); top3.append(t in seen)
    return np.array(top1), np.array(top3), S.max(axis=1), order[:, 0]

fids = train.fact_id.values
Mv = eval_ret.score_matrices(val.question.tolist());  Mt =
    eval_ret.score_matrices(test.question.tolist())
Ov = eval_ret.score_matrices(ood_val);                Ot = eval_ret.score_matrices(ood_test)

# ---- (a) experimental weight search on VALIDATION (grid, step 0.1) ----
grid = []
for ww in np.arange(0, 1.01, 0.1):
    for wc in np.arange(0, 1.01 - ww, 0.1):
        ws = max(0.0, round(1 - ww - wc, 2))
        if not eval_ret.use_dense and ws > 0: continue
        t1, t3, _, _ = rank_metrics(eval_ret.combine(*Mv, (ww, wc, ws)), val.fact_id.values, fids)
        grid.append(dict(w_word=round(ww,1), w_char=round(wc,1), w_sem=ws, top1=t1.mean(),
            top3=t3.mean()))
grid = pd.DataFrame(grid).sort_values(["top1","top3"], ascending=False).reset_index(drop=True)
print("Top 8 weight settings on validation:"); display(grid.head(8))
BEST_W = tuple(grid.loc[0, ["w_word","w_char","w_sem"]].astype(float))
print("Selected weights (w_word, w_char, w_sem):", BEST_W)

# ---- (b) single-method baselines vs hybrid (validation and test) ----
cands = {"Word only":(1,0,0), "Char only":(0,1,0), "MiniLM only":(0,0,1), "Equal
    hybrid":(1/3,1/3,1/3), "Tuned hybrid":BEST_W}
rows = []
for name, w in cands.items():
```


Scanned with OKEN Scanner

```
    if name == "MiniLM only" and not eval_ret.use_dense: continue
    for split_name, M, d in [("val", Mv, val), ("test", Mt, test)]:
        t1, t3, _, _ = rank_metrics(eval_ret.combine(*M, w), d.fact_id.values, fids)
        rows.append(dict(method=name, split=split_name, top1=t1.mean(), top3=t3.mean()))
cmp_df = pd.DataFrame(rows); display(cmp_df.pivot(index="method", columns="split",
    values=["top1","top3"]).round(4))
sns.barplot(data=cmp_df, x="method", y="top1", hue="split"); plt.xticks(rotation=20); plt.ylim(0.5,
    1.02); plt.title("Top-1 accuracy by retrieval method"); plt.show()

# ---- (c) confidence-threshold sweep on VALIDATION (answerable vs unrelated queries) ----
Sv = eval_ret.combine(*Mv, BEST_W); So = eval_ret.combine(*Ov, BEST_W)
c1, c3, conf_in, _ = rank_metrics(Sv, val.fact_id.values, fids)
conf_out = So.max(axis=1)

def threshold_report(t, conf_in, correct_in, conf_out):
    acc_in, acc_out = conf_in >= t, conf_out >= t
    TP, FN, FP, TN = acc_in.sum(), (~acc_in).sum(), acc_out.sum(), (~acc_out).sum()
    prec = TP/(TP+FP) if TP+FP else 0; rec = TP/(TP+FN); f1 = 2*prec*rec/(prec+rec) if prec+rec else
         0
    return dict(threshold=round(t,3), precision=prec, recall=rec, f1=f1, rejection_acc=TN/(TN+FP),
                answered_correct=(acc_in & correct_in).sum()/len(conf_in))

sweep = pd.DataFrame([threshold_report(t, conf_in, c1, conf_out) for t in np.arange(0.20, 0.96,
    0.025)])
BEST_T = float(sweep.sort_values(["f1","rejection_acc"], ascending=False).iloc[0].threshold)
print("Validation sweep (selected rows):"); display(sweep.iloc[::3].round(3))
print(f"Selected threshold on validation = {BEST_T}  (the slide value 0.35 gives:)")
display(pd.DataFrame([threshold_report(0.35, conf_in, c1, conf_out)]).round(3))
plt.figure(figsize=(8,4));
for col in ["precision","recall","f1","rejection_acc"]: plt.plot(sweep.threshold, sweep[col],
    label=col)
plt.axvline(BEST_T, ls="--", c="k"); plt.xlabel("confidence threshold"); plt.legend();
    plt.title("Threshold selection on validation set"); plt.show()
sns.kdeplot(conf_in, label="answerable (val)", fill=True); sns.kdeplot(conf_out, label="unrelated
    (val)", fill=True); plt.xlabel("confidence"); plt.legend(); plt.show()
```

Section 8 - Build the Chatbot

```
class CollegeChatbot:
    def __init__(self, df, weights, threshold):
        self.df = df.reset_index(drop=True); self.threshold = threshold; self.weights = weights
        self.ret = HybridRetriever(weights).fit(self.df.question.tolist())
    def answer(self, query, k=3):
        if not isinstance(query, str) or not query.strip():
            return dict(found=False, answer="Please type a question.", matched_question="-",
                confidence=0.0, category="-", top=[])
        top, sc = self.ret.search(query, k, self.weights); r = self.df.iloc[top[0]]
        res = dict(found=bool(sc[0] >= self.threshold), confidence=float(sc[0]),
            matched_question=r.question, category=r.category,
                top=[(self.df.question[i], float(s)) for i, s in zip(top, sc)])
        res["answer"] = r.answer if res["found"] else NOT_FOUND_MSG
        if not res["found"]: res["category"] = "-"
        return res

bot = CollegeChatbot(kb, BEST_W, BEST_T)
for q in ["What is the attendance policy?", "Can I write the exam with 72% attendance?", "Explain
    the SGPA calculation", "What is the weather today?"]:
    r = bot.answer(q); print(f"\nQ: {q}\nFOUND={r['found']} conf={r['confidence']:.2f}
        cat={r['category']}\nA: {r['answer'][:230]}")

try:                                    # lets widgets display in new Colab versions
    from google.colab import output; output.enable_custom_widget_manager()
except Exception: pass
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output
import html as _html

def build_ui(bot):
    history = widgets.Output(layout=widgets.Layout(border="1px solid #cbd5e1", padding="8px",
        max_height="460px", overflow_y="auto"))
    box = widgets.Text(placeholder="Ask about attendance, exams, SGPA, fees, first-semester
        syllabus...", layout=widgets.Layout(width="70%"))
    btn = widgets.Button(description="Ask", button_style="primary", icon="search")
    examples = ["What is the attendance policy?", "What is the minimum attendance required for
        semester exams?", "Explain the SGPA calculation",
                "What are the subjects in the first semester?", "What is Unit 1 in my mathematics
                    syllabus?", "What is the revaluation procedure?"]
    def show(q):
        try:
            r = bot.answer(q); e = _html.escape
            color = "#e8f7ee" if r["found"] else "#fdecea"
            meta = (f"<b>Matched KB question:</b> {e(r['matched_question'])}<br><b>Category:</b>
                {e(r['category'])}  |  <b>Confidence:</b> {r['confidence']:.2f}"
                    if r["found"] else f"<b>Confidence:</b> {r['confidence']:.2f} (below threshold
                        {bot.threshold})")
            with history:
                display(HTML(f"<div
```


Scanned with OKEN Scanner

```
                    style='background:#e0e7ff;padding:8px;border-radius:8px;margin:6px 0'>
                    <b>{e(q)}</b></div>"
                             f"<div
                                 style='background:{color};padding:10px;border-radius:8px;margin:6px
                                  0 14px'> {e(r['answer'])}<hr style='margin:6px
                                 0'><small>{meta}</small></div>"))
        except Exception as ex:
            with history: print("Something went wrong:", ex)
    def on_ask(_):
        q = box.value.strip()
        if q: show(q); box.value = ""
    btn.on_click(on_ask)
    try: box.on_submit(on_ask)          # only exists in older ipywidgets; Enter key is optional,
        the Ask button always works
    except Exception: pass
    chips = []
    for ex in examples:
        b = widgets.Button(description=ex, layout=widgets.Layout(width="auto")); b.on_click(lambda
            _, t=ex: show(t)); chips.append(b)
    display(HTML("<h2 style='color:#1e3a8a'> College Regulations & Syllabus Assistant</h2><p>Hello!
        Ask me about attendance, examinations, grading, fees, hostel, library and the
        <b>first-semester B.Tech syllabus</b>. "
                 "<i>(Prototype data is SAMPLE - confirm with the Academic Office.)</i></p><b>Try an
                      example:</b>"))
    display(widgets.VBox([widgets.HBox(chips[:3]), widgets.HBox(chips[3:])]), widgets.HBox([box,
        btn]), history)
build_ui(bot)

# ---- Simple fallback that ALWAYS works (no widgets needed) ----
def ask(question):
    r = bot.answer(question)
    print("Q:", question); print("A:", r["answer"])
    if r["found"]: print(f"   [category: {r['category']} | confidence: {r['confidence']:.2f}]")
    else: print(f"   [confidence {r['confidence']:.2f} below threshold {bot.threshold}]")
    print()
ask("What is the minimum attendance required?")

# ---- Chat in the notebook (no widgets needed). Type 'exit' to stop. ----
while True:
    q = input("You: ").strip()
    if q.lower() in {"exit", "quit", "q", ""}: print("Bye!"); break
    ask(q)
```


Scanned with OKEN Scanner

Section 9 - Testing

```
T = [  # (query, expected category, key phrase expected in the answer, type)
("What is the attendance policy?","Attendance","75%","exact"),("What is the minimum attendance
    required?","Attendance","75%","exact"),
("How much attendance is compulsory?","Attendance","75%","paraphrase"),("Is 75% attendance
    mandatory?","Attendance","75%","exact"),
("Can I write semester exams with 72% attendance?","Attendance","condonation","exact"),("I have 72%
    attendance can i write exam","Attendance","condonation","paraphrase"),
("What happens if my attendance is below 75%?","Attendance","condon","exact"),("What is attendance
    condonation?","Attendance","condonation","exact"),
("min attendence for sem exams","Attendance","75%","spelling"),("attendence
    rules","Attendance","75%","spelling"),
("Is attendance compulsory for mid exams?","Attendance","mid","paraphrase"),("how to calculate
    attendance percentage","Attendance","100","paraphrase"),
("attendance","Attendance","75%","short"),
("Is attendance calculated separately for every subject?","Attendance","separately","exact"),
("Can medical certificate help if my attendance is short?","Attendance","medical","paraphrase"),
("Does attendance affect internal marks?","Attendance","internal","exact"),
("What is the revaluation procedure?","Examination","revaluation","exact"),("how to apply for
    reval","Examination","revaluation","short"),
("revalution fee","Examination","1,000","spelling"),("What is the difference between recounting and
    revaluation?","Examination","recount","paraphrase"),
("What are the malpractice rules in exams?","Examination","malpractice","exact"),("Can I take my
    mobile phone into the exam hall?","Examination","mobile","paraphrase"),
("How do I download my hall ticket?","Examination","hall ticket","paraphrase"),("what are
    supplementary exams","Examination","supplementary","short"),
("I failed a subject, how do I clear the backlog?","Examination","backlog","long"),("What is the
    pass mark for a theory subject?","Examination","28","paraphrase"),
("How are internal marks calculated?","Examination","30","paraphrase"),("what is the improvement
    exam","Examination","improvement","short"),
("Explain the SGPA calculation","Academic Regulations","SGPA","exact"),("how to calculate
    cgpa","Academic Regulations","CGPA","paraphrase"),
("What is the grading system?","Academic Regulations","A+","exact"),("what is academic
    probation","Academic Regulations","probation","short"),
("What is detention in college?","Academic Regulations","detain","paraphrase"),("How many credits
    are needed for B.Tech?","Academic Regulations","160","paraphrase"),
("What are mandatory courses?","Academic Regulations","non-credit","exact"),
("How do I get a bonafide certificate?","College Administration","bonafide","exact"),("how to get
    transfer certificate","College Administration","transfer certificate","paraphrase"),
("What is the hostel in time?","College Administration","8:00 PM","exact"),("how many books can i
    borrow from library","College Administration","3 books","paraphrase"),
("How do I apply for scholarships?","College Administration","scholarship","exact"),("I forgot my
    student portal password","College Administration","password","paraphrase"),
("How can I pay my college fees online?","College Administration","portal","paraphrase"),
("I lost a library book, what should I do?","Practical Student Problems","replace","long"),("What if
     I miss a mid exam due to illness and want a re-test?","Practical Student
    Problems","HOD","long"),
("I missed the lab session, what now?","Practical Student Problems","make-up","paraphrase"),("Where
    can I see the academic calendar?","Practical Student Problems","calendar","exact"),
("What are the subjects in the first semester?","Syllabus","Mathematics-I","exact"),("1st sem
    subjects","Syllabus","Mathematics-I","short"),
("What is Unit 1 in my mathematics syllabus?","Syllabus","Unit 1","exact"),("unit 3 of
    physics","Syllabus","Unit 3","short"),
("what is unit 5 in c programming","Syllabus","Unit 5","paraphrase"),("How many credits are there in
     first semester?","Syllabus","Total","exact"),
("What laboratory subjects are there in first semester?","Syllabus","Laborator","paraphrase"),("What
     is Cayley-Hamilton theorem?","Syllabus","characteristic","exact"),
("what is numerical aperture","Syllabus","NA","short"),
("What is the weather today?","REJECTED","","unrelated"),("Who won the cricket match
    yesterday?","REJECTED","","unrelated"),
("How do I bake a cake?","REJECTED","","unrelated"),("What is the syllabus of Operating
    Systems?","REJECTED","","unrelated"),
("Tell me a joke","REJECTED","","unrelated"),("Which movie should I watch
    tonight?","REJECTED","","unrelated"),
]
rows = []
for q, ecat, key, kind in T:
    r = bot.answer(q); pred = r["category"] if r["found"] else "REJECTED"
    ok = (pred == "REJECTED") if ecat == "REJECTED" else (pred == ecat and key.lower() in
        r["answer"].lower())
    rows.append(dict(query=q, type=kind, expected_category=ecat, retrieved_category=pred,
        retrieved_answer=r["answer"][:110]+"...", confidence=round(r["confidence"],3), correct=ok))
test_df = pd.DataFrame(rows)
pd.set_option("display.max_colwidth", 120); display(test_df)
print(f"\nManual test accuracy: {test_df.correct.mean():.1%}
    ({test_df.correct.sum()}/{len(test_df)})")
display(test_df.groupby("type").correct.agg(["mean","count"]).round(3))
```


Scanned with OKEN Scanner

Section 10 - Results and Discussion

```
# ---- Held-out TEST evaluation ----
St = eval_ret.combine(*Mt, BEST_W); So_t = eval_ret.combine(*Ot, BEST_W)
t1, t3, conf_t, best_idx = rank_metrics(St, test.fact_id.values, fids); conf_o = So_t.max(axis=1)
rep = threshold_report(BEST_T, conf_t, t1, conf_o)
res = pd.DataFrame({"metric":["Top-1 accuracy (answerable)","Top-3 accuracy
    (answerable)","Answer-vs-reject precision","Answer-vs-reject recall","Answer-vs-reject
    F1","Rejection accuracy (unrelated)","Answered AND correct (answerable)"],
                    "value":[t1.mean(), t3.mean(), rep["precision"], rep["recall"], rep["f1"],
                        rep["rejection_acc"], rep["answered_correct"]]})
display(res.round(4)); print(f"Weights = {BEST_W} | threshold = {BEST_T} | test queries: {len(test)}
     answerable + {len(ood_test)} unrelated")

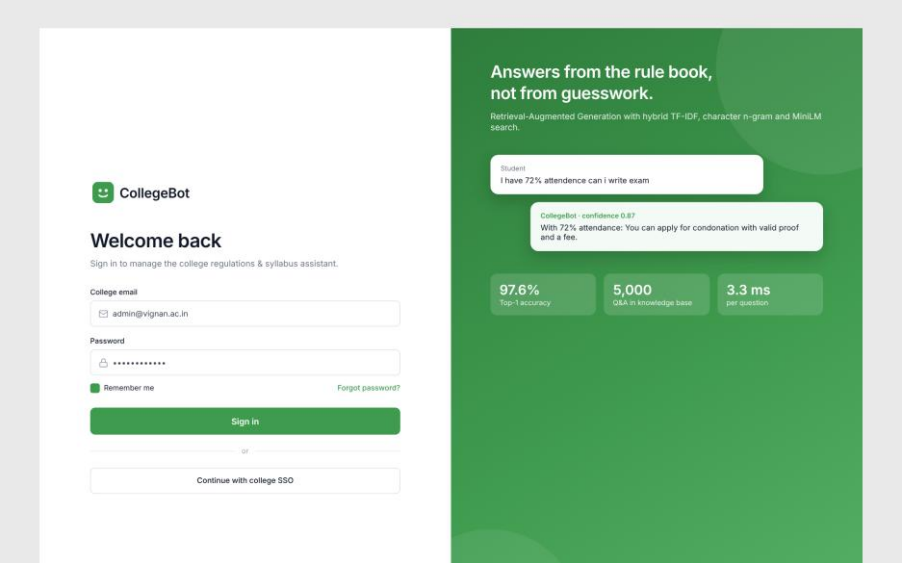
# ---- Response time on the deployed chatbot ----
qs = test.question.sample(min(100, len(test)), random_state=SEED).tolist(); t0 = time.perf_counter()
for q in qs: bot.answer(q)
print(f"Average response time: {(time.perf_counter()-t0)/len(qs)*1000:.1f} ms per query")

# ---- Category-level confusion matrix (test + unrelated, after thresholding) ----
true_lab = list(test.category) + ["OUT_OF_KB"] * len(ood_test)
pred_lab = [train.category[j] if c >= BEST_T else "REJECTED" for j, c in zip(best_idx, conf_t)] + \
           [train.category[j] if c >= BEST_T else "REJECTED" for j, c in zip(So_t.argmax(axis=1),
               conf_o)]
labs_t = sorted(set(true_lab)); labs_p = sorted(set(pred_lab) | set(train.category))
cm = confusion_matrix(true_lab, pred_lab, labels=sorted(set(labs_t) | set(labs_p)))
plt.figure(figsize=(9,7)); sns.heatmap(pd.DataFrame(cm, index=sorted(set(labs_t)|set(labs_p)),
    columns=sorted(set(labs_t)|set(labs_p))), annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted category"); plt.ylabel("True category"); plt.title("Confusion matrix (test
    split)"); plt.show()

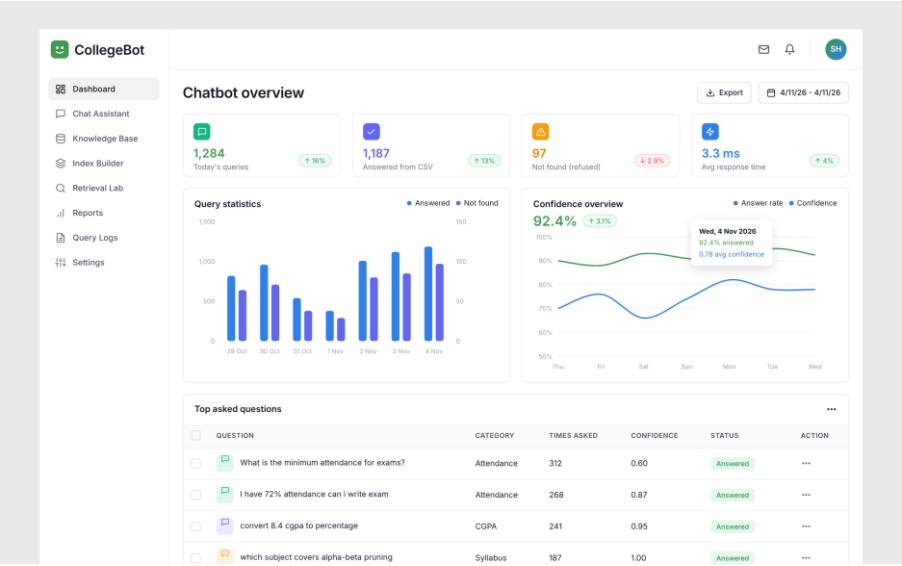
# ---- Error analysis: show wrong top-1 retrievals ----
wrong = test[~t1].copy(); wrong["retrieved_question"] = [train.question[j] for j in best_idx[~t1]];
    wrong["retrieved_category"] = [train.category[j] for j in best_idx[~t1]]
print(f"Wrong top-1 on test: {len(wrong)}");
    display(wrong[["question","category","retrieved_question","retrieved_category"]].head(10))
fp = [(q, round(float(c),3)) for q, c in zip(ood_test, conf_o) if c >= BEST_T]; print("Unrelated
    queries wrongly answered:", fp)

# ---- Save the indexed artifacts of the deployed chatbot ----
import joblib
os.makedirs("rag_artifacts", exist_ok=True)
joblib.dump({"word": bot.ret.word_vec, "char": bot.ret.char_vec, "weights": BEST_W, "threshold":
    BEST_T}, "rag_artifacts/vectorizers.joblib")
kb.drop(columns=["split"]).to_csv("rag_artifacts/knowledge_base_used.csv", index=False)
if bot.ret.use_dense:
    np.save("rag_artifacts/question_embeddings.npy", bot.ret.E)
    if HAS_FAISS: faiss.write_index(bot.ret.index, "rag_artifacts/faiss.index")
print("Saved:", os.listdir("rag_artifacts"))
```

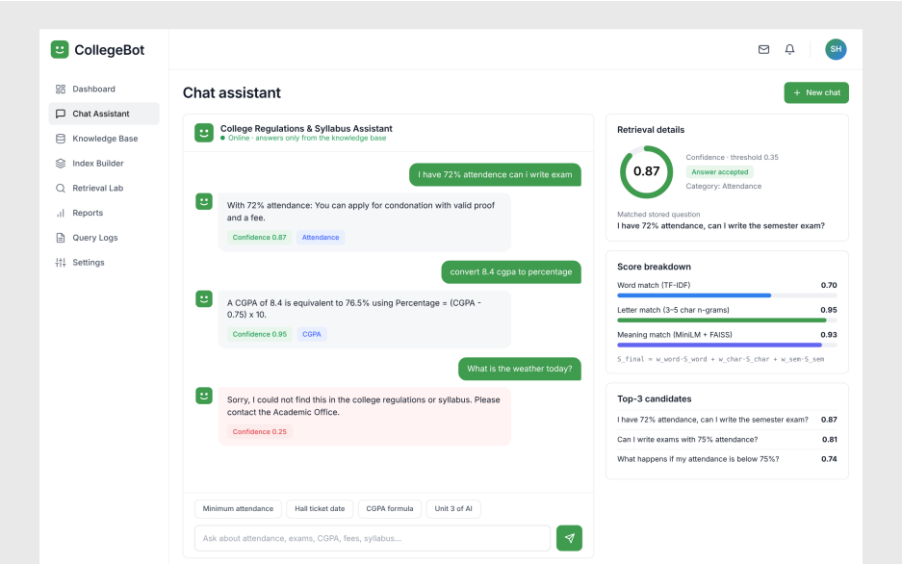

Scanned with OKEN Scanner



Screen 01 Login

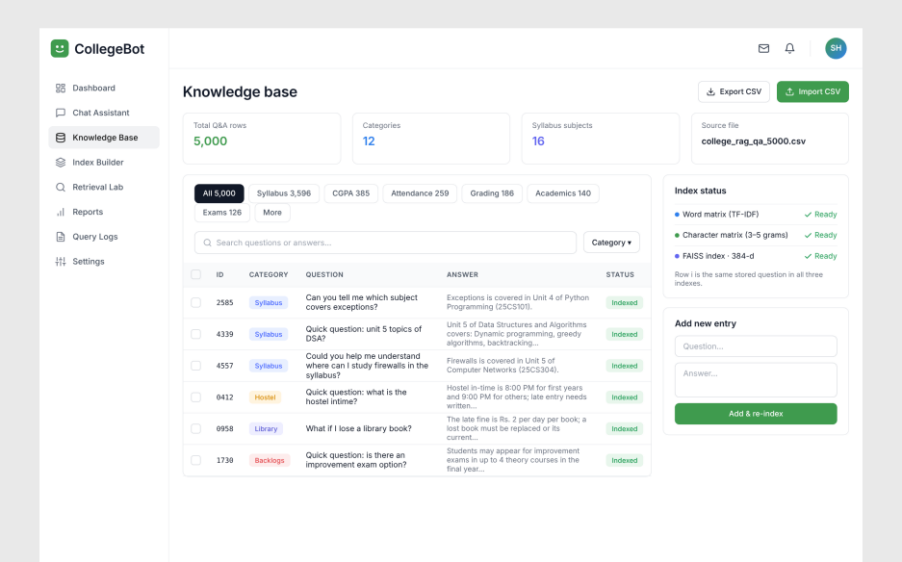


Screen 02 Dashboard - chatbot overview

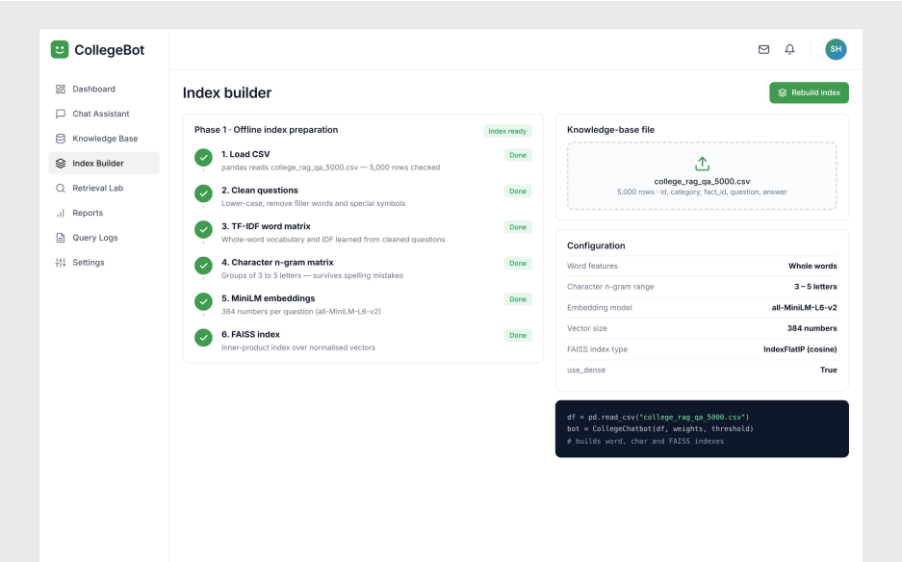


Screen 03 Chat assistant with retrieval details

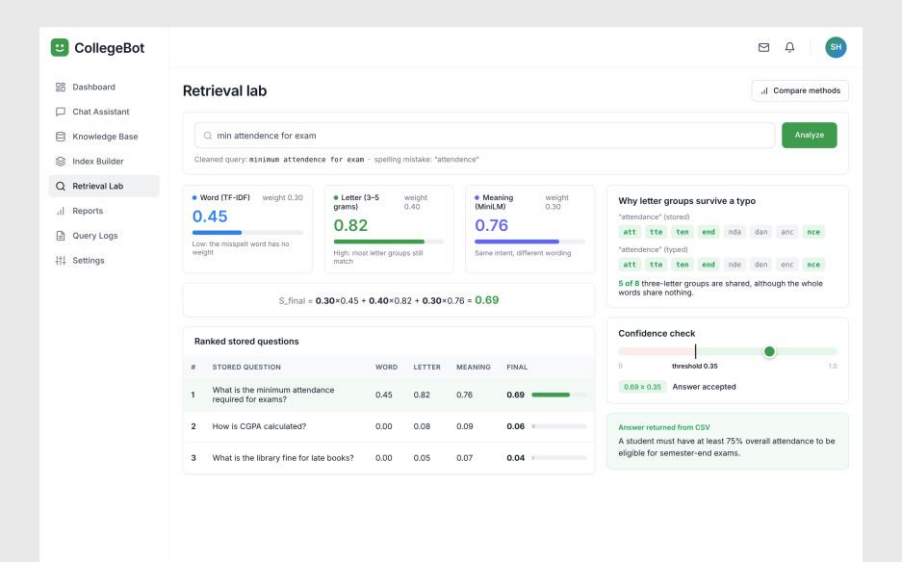
Scanned with OKEN Scanner



Screen 04 Knowledge base (5,000 Q&A)

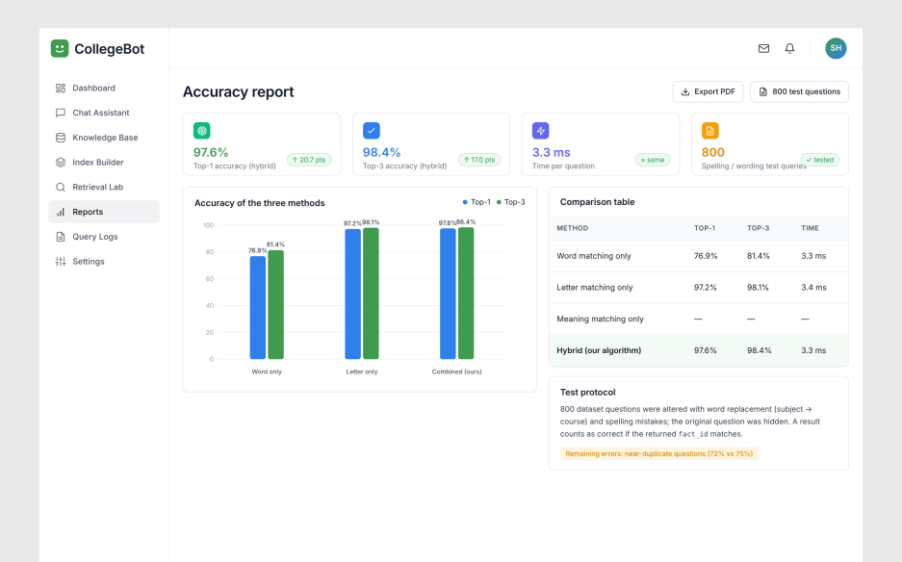


Screen 05 Index builder - Phase 1

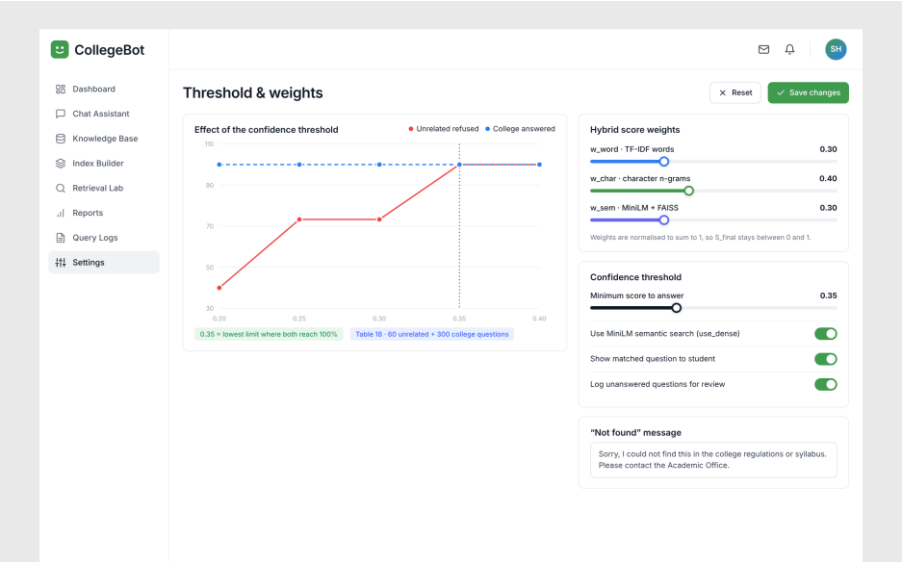


Screen 06 Retrieval lab - hybrid score breakdown

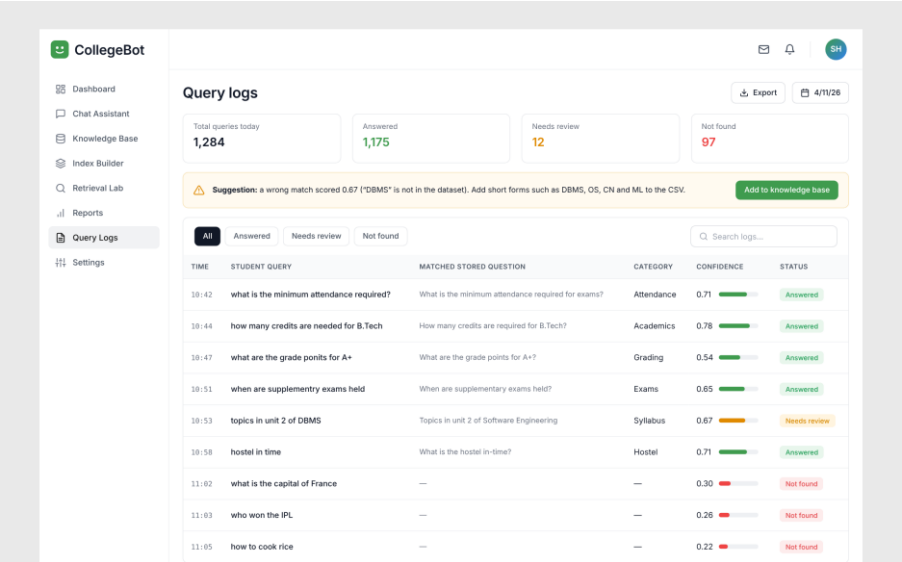
Scanned with OKEN Scanner



Screen 07 Accuracy report

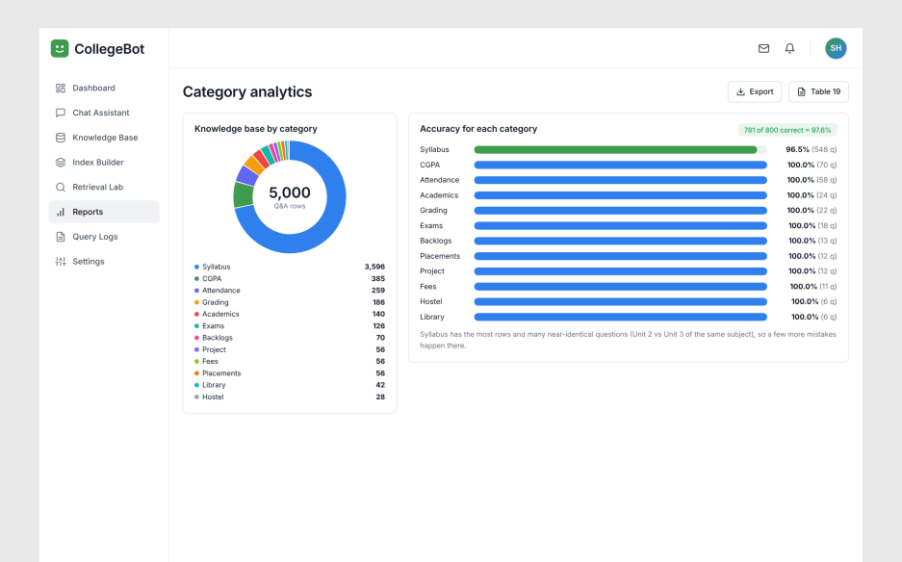


Screen 08 Threshold and weights settings



Screen 09 Query logs and unanswered questions

Scanned with OKEN Scanner



Screen 10 Category analytics

Scanned with OKEN Scanner